

MVP RESEARCH REPORT

MVP 종합보고서

Unitree Go2

미경험 험지 적응 시뮬레이션 및
실기 자율주행 프로젝트

미경험 지형의 보행 정책 일반화와 명령 수행 능력 평가

프로젝트 기간	2026.07 ~ 2026.12.11	팀명	FOOTHOLD
MVP 중간발표	2026.09.30	팀장	오흥재
결과 기준일	2026.09.29	팀원	임석현 · 오현민 · 맹라현 · 이민우

RESEARCH OVERVIEW

험지 통과와 명령 수행을 함께 평가한다

Unitree Go2의 미경험 험지 적응을 평가하고, 전진 보행 성능의 개선이 기본 명령 수행 능력과 어떻게 연결되는지 살폈다. 지형 통과와 명령 응답을 두 평가 축으로 나누고, 학습 지형·명령·보상 변경의 결과를 비교했다.

학습 여부를 다시 분류한 미경험 8종의 성능을 중심으로 결과를 제시한다. 성공률의 개선과 함께 속도 추종 실패, 회전 낙상, 희소 발판의 한계를 기록하며, 배포 모델의 성능과 아직 충족하지 못한 기준을 구분한다.

미경험 8종 · 세 속도 평균

43.9 → 88.5%

NVIDIA 기준선 → foothold-v2 · 난이도 0.5

보고서가 집중한 두 축

험지 통과 명령 수행

성공률의 증가와 남은 실패를 함께 기록

MVP 공식 제출본 · 결과 기준 2026.09.29

HTML은 영상을 재생하며, PDF는 실제 영상의 주요 장면과 원본 링크를 제공한다.

보고서의 흐름

- | | |
|---------------------------|---------------------------|
| 00 연구 개요와 핵심 결과 | 01 평가 범위 · 학습과 미경험 지형의 구분 |
| 02 미경험 지형 일반화 성능 | 03 성공률을 구성하는 네 조건 |
| 04 지형별 성능 | 05 난이도 변화에 따른 성능 |
| 06 연구 과정 · 문제, 가설, 실험과 검증 | 07 학습 방법과 레시피 |
| 08 탐색 중단 기준의 기준과 한계 | 09 명령 수행 능력 평가 |
| 10 배포 모델과 참고 비교군 | 11 관측과 정책의 다음 연구 과제 |
| 12 재현 방법 | 13 영상으로 확인하는 결과와 한계 |
| 14 근거 자료 | |

0. 연구 개요와 핵심 결과

foothold-v2 로 v2g2-feetair01 iter3000 을 배포한다. 이 문서는 그 판이 무엇을 할 수 있고 무엇을 못 하는지, 그리고 어떤 물음을 거쳐 거기에 닿았는지를 적는다.

- 학습에 없던 여덟 지형에서 NVIDIA 43.9 % -> v2 88.5 %
- 그 중 일곱은 99.7 % 이상이고 stepping_stones 하나만 8.0 % 다
- 정책 수준(보상·명령)의 탐색은 멈출 선에 닿았다. 다음은 관측이다

배포 기준을 다 채운 것은 아니다. 축 2 아홉 칸 중 여덟이다. 그런데도 이것을 2 차 배포로 내세우는 까닭은 (1) 축 1 이 네 체크포인트 전부에서 진짜 하락 0 이고 (2) 축 2 를 처음으로 재서 남은 한 칸을 이름으로 적을 수 있게 되었기 때문이다. 남은 칸은 다음 실험의 목표다.

1. 평가 범위 · 학습과 미경험 지형의 구분

평가 지형 열여섯 종 중 열 종을 우리는 unseen10 이라고 불러 왔다. 그런데 v2 에게는 그 말이 틀리다. 학습 설정을 열어서 확인했다.

params/env.yaml sub_terrains (2026-09-28 · 학습 로그에서 직접 읽음)

NVIDIA 배포본 6종 rough6

foothold-v1 7종 rough6 + forward_gap

v2 (v2g2) 8종 rough6 + omni_gap + rails

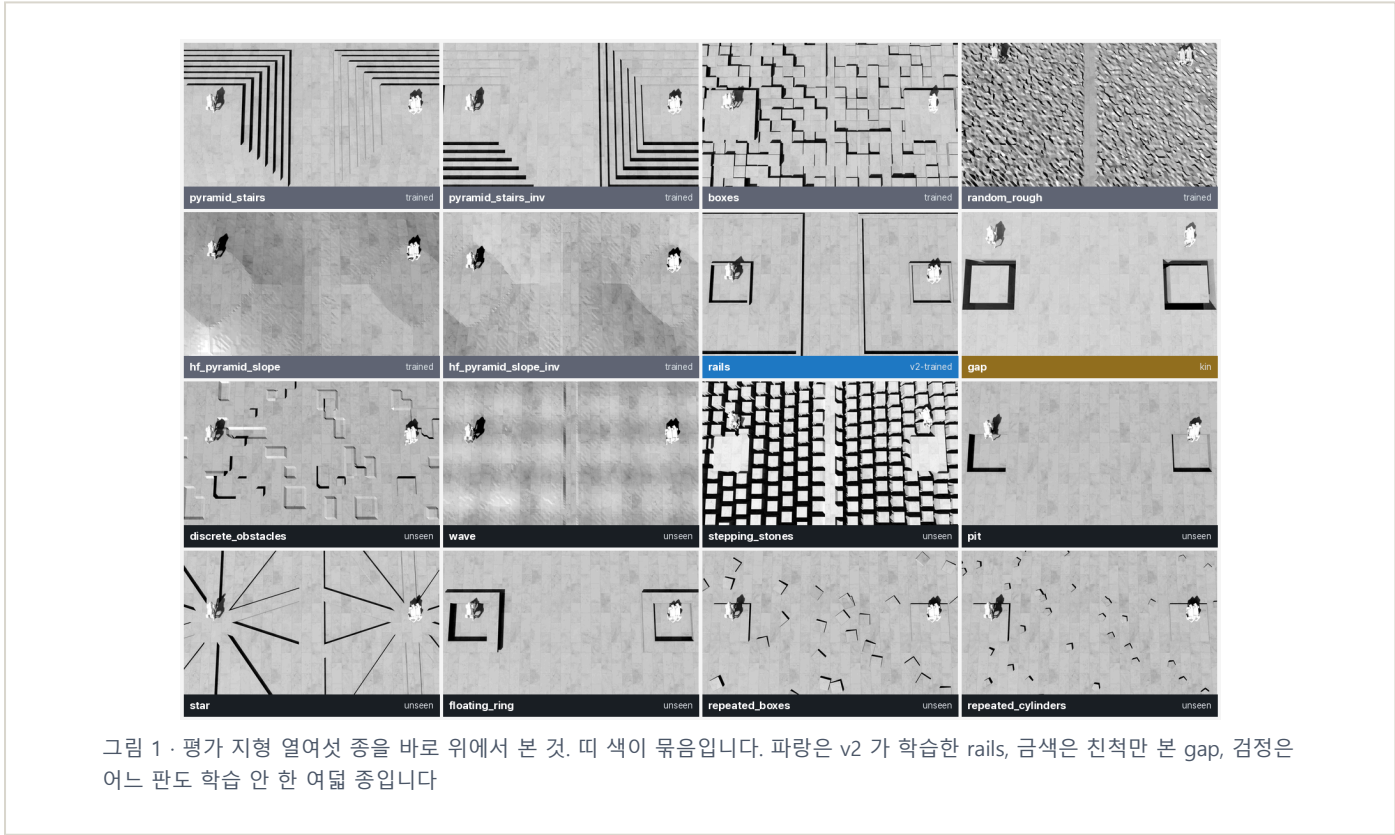
rails 는 평가 목록에 있는 바로 그 지형이고 v2 가 학습에 넣었다. 함수도 인자도 같다 (mesh_terrains:rails_terrain · thickness (0.08, 0.18) · height (0.05, 0.18) · platform 1.5). 미경험이 아니다.

gap 은 사정이 다르다. 평가는 MeshGapTerrainCfg 이고 v2 학습은 omni_gap_terrain(mode=ring) 이다. 함수가 다르다. 폭 범위만 같다. 그래서 「친척을 봤다」로 따로 센다.

그래서 이 보고서는 숫자를 네 묶음으로 나눠 적고, 머리기사는 여덟 종으로 쓴다.

묶음	지형	뜻
어느 판도 학습 안 한 8종	discrete_obstacles · floating_ring · pit · repeated_boxes · repeated_cylinders · star · stepping_stones · wave	머리기사는 이것으로 쓴다
친척 학습 1종	gap	v1 은 forward_gap, v2 는 omni_gap
v2 학습 1종	rails	v2 에게는 학습 지형이다
셋 다 학습 6종	rough6	NVIDIA 가 연습한 지형

네 묶음이 실제로 어떻게 생겼나



한 칸은 타일 2 x 2 이고 로봇 넷이 크기 자를 겸한다. 배포본 v2g2-feetair01 iter3000 을 같은 카메라로 태워 찍었다.

대비를 폼다. 원본은 밝기가 184 ~ 191 에 몰려 있어 일곱 종이 거의 안 읽혔다 (홀어짐 9.7 ~ 15.4). 재질이 균일한 밝은 회색이고 조명이 부드러워서다. 카메라를 45 도로 기울여도 안 고쳐졌다 (9.7 -> 8.6). 각도가 아니라 조명 문제였다. 그래서 **휘도만** 백분위 0.5 ~ 99.5 로 폼다. 채널마다 퍼면 없는 색이 생긴다 (hf_pyramid_slope_inv 가 보라색이 됐다).

퍼는 것은 표시 방식이지 없는 요철을 만드는 것이 아니다. 원본 스틸과 퍼기 전후 값은 `sim/eval/results/20260928-terrain-shots/stills/index.json` 에 그대로 있다.

2. 미경험 지형 일반화 성능

v2 는 처음 보는 지형에서 무엇을 얻었나

난이도 0.5 · 지형마다 속도마다 100판 · 값은 성공률 %

■ NVIDIA ■ foothold-v1 ■ v2 (v2g2 @3000)

평가 전체 48 칸



지형 16 x 속도 3

48 칸

셋 다 학습한 6종



NVIDIA 가 연습한 지형

18 칸

어느 판도 학습 안 한 8종



처음 보는 지형

24 칸

그 중 7종 (stepping_stones 뺀)



v2 가 사실상 푼 것

21 칸

rails 는 v2 가 학습에 넣은 지형이라 «어느 판도 학습 안 한» 묶음에서 뺐다 · gap 은 v1·v2 가 친척(forward_gap · omni_gap)을 학습해서 뺐다

그림 2 · 네 묶음에서 세 판의 성공률. 어느 판도 학습 안 한 여덟 지형에서 NVIDIA 43.9 %가 v2 88.5 %가 됐습니다

난이도 0.5 · 지형마다 속도마다 100 판. 칸은 지형 x 속도 셋이다 (8종이면 $8 \times 3 = 24$ 칸). 곧 아래 값은 세 속도(0.5 · 1.0 · 1.5 m/s)의 평균이다.

허브 첫 화면 카드와 수가 다른 까닭. 그 카드는 1.0 m/s 한 속도만 켜다 (카드에 조건이 적혀 있다). 같은 여덟 종이 거기서는 62 -> 90 이고 여기서는 43.9 -> 88.5 다. 둘 다 맞고 재는 범위가 다르다.

속도별로 퍼 보면 까닭이 한 줄이다. 여덟 중 난이도 0.5 실측이다.

속도	NVIDIA	v2
0.5 m/s	60.9	87.5
1.0 m/s	62.0	90.5
1.5 m/s	8.8	87.4
세 속도 평균	43.88	88.46

NVIDIA 만 1.5 m/s 에서 무너진다 (8.8 %). 그래서 세 속도 평균에서 차이가 +44.6 %p 로 벌어지고, 1.0 m/s 만 보면 +28.5 %p 다. 다만 1.5 m/s 는 NVIDIA 의 학습 명령 범위 밖이므로 그 사실을 같이 읽어야 한다 (3 절).

묶음	칸	NVIDIA	foothold-v1	v2
평가 전체	48	41.17	88.88	94.19
셋 다 학습한 6종	18	50.78	99.33	100.00
어느 판도 학습 안 한 8종	24	43.88	86.71	88.46
그 중 7종 (stepping_stones 뺀)	21	50.14	99.10	99.95

읽는 법이 셋이다.

1. NVIDIA 대비는 크다. 여덟 종에서 43.88 -> 88.46, +44.58 %p 다
2. v1 대비는 작다. 86.71 -> 88.46, +1.75 %p 다. 이것은 시드만 바뀌도 생기는 흔들림 1.25 %p 와 같은 크기다 (8 절)
3. 여덟 종 평균 88 을 끌어내리는 것은 stepping_stones 하나다. 그것을 빼면 99.95 % 다

2-1. 눈으로 먼저 · 목적성 세 화면

숫자를 보기 전에 화면으로 본다. 주제가 「미경험 험지 지형 정책 적응」이니 어느 판도 학습 안 한 8 종 안에서 세 칸을 골랐다. 기준을 먼저 적고 그 기준으로 골랐다.

고른 칸	NVIDIA -> v1 -> v2	왜 이 칸인가
floating_ring 1.5 m/s	0 -> 87 -> 99	세 판의 값이 다 다른 유일한 칸 · 계보 전체가 한 화면에 들어온다
pit 1.5 m/s	0 -> 100 -> 100	NVIDIA 가 아예 못 하는 칸
stepping_stones 1.0 m/s	0 -> 0 -> 24	아직 못 하는 칸 · v2 가 v1 을 넘은 가장 큰 칸

gap 과 rails 는 v2 에게 학습 지형이라 뺐다 (1 절의 네 묶음). 잘하는 칸만 걸면 자기 참조가 된다. 그래서 못 하는 칸을 같이 넣었다.

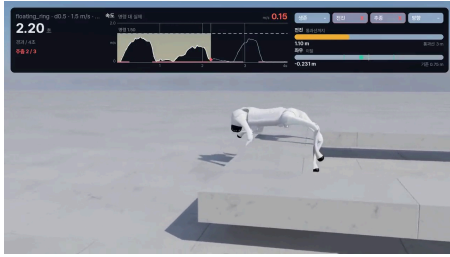
컷은 num_envs 1 한 판의 예시다 확인됨 (sim/eval/gallery_build.py 머리말 · 「영상은 성공률이 아니다」). 옆에 적은 성공률은 지형·속도마다 100 판 으로 따로 켜 값이다. 한 판이 그 100 판을 대표한다고 읽지 말 것.

울타리 안에서 컷 길이가 다르다 (예 floating_ring 1.5 는 4.00 · 3.80 · 3.72 초). 한 판이 끝나는 시점이 판마다 달라서다. 왜 다른지는 안 켜다 이므로 길이로 성공·실패를 읽지 말 것.

floating_ring · 1.5 m/s · 세 판이 다 다르다

볼 것: 고리를 넘는가. NVIDIA 는 0 %, foothold-v1 은 87 %, foothold-v2 는 99 % 다.

NVIDIA



100 판 성공률 0 % · 이 컷은 4.00 초 한 판

V01 · 영상 보기 · 대표 장면 2.2초 / 4초

지형을 마주한 보행 장면. 이 한 판의 모습과 캡션의 100회 평가 성공률을 구분한다.

foothold-v1



100 판 성공률 87 % · 이 컷은 3.80 초 한 판

V02 · 영상 보기 · 대표 장면 2.2초 / 3.8초

지형을 마주한 보행 장면. 이 한 판의 모습과 캡션의 100회 평가 성공률을 구분한다.

foothold-v2



100 판 성공률 99 % · 이 컷은 3.72 초 한 판

V03 · 영상 보기 · 대표 장면 2.2초 / 3.72초

지형을 마주한 보행 장면. 이 한 판의 모습과 캡션의 100회 평가 성공률을 구분한다.

pit · 1.5 m/s · NVIDIA 가 아예 못 한다

볼 것: 구덩이를 건너는가. NVIDIA 0 % · v1 100 % · v2 100 %. 이 칸은 foothold-v1 이 이미 풀었다. v2 가 더한 것이 없다.

NVIDIA



100 판 성공률 0 % · 이 컷은 4.00 초 한 판

V04 · 영상 보기 · 대표 장면 2.2초 / 4초

지형을 마주한 보행 장면. 이 한 판의 모습과 캡션의 100회 평가 성공률을 구분한다.

foothold-v1



100 판 성공률 100 % · 이 컷은 3.74 초 한 판

V05 · 영상 보기 · 대표 장면 2.2초 / 3.74초

지형을 마주한 보행 장면. 이 한 판의 모습과 캡션의 100회 평가 성공률을 구분한다.

foothold-v2



100 판 성공률 100 % · 이 컷은 3.68 초 한 판

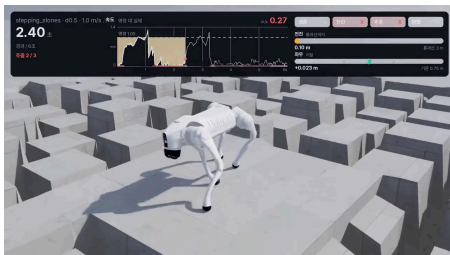
V06 · 영상 보기 · 대표 장면 2.2초 / 3.68초

지형을 마주한 보행 장면. 이 한 판의 모습과 캡션의 100회 평가 성공률을 구분한다.

stepping_stones · 1.0 m/s · 아직 못 하는 칸

볼 것: 디딤돌 사이에서 발을 지지하는가. 1.0 m/s에서 NVIDIA와 foothold-v1의 종합 성공률은 0%, foothold-v2는 24%다. 0.5와 1.5 m/s에서는 세 모델 모두 0%다. 한 영상의 통과 여부와 100회 평가의 성공률은 구분한다.

NVIDIA

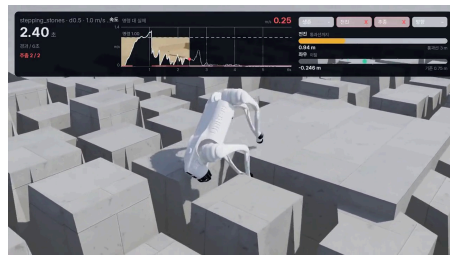


100 판 성공률 0 % · 이 컷은 6.00 초 한 판

V07 · 영상 보기 · 대표 장면 2.4초 / 6초

지형을 마주한 보행 장면. 이 한 판의 모습과 캡션의 100회 평가 성공률을 구분한다.

foothold-v1

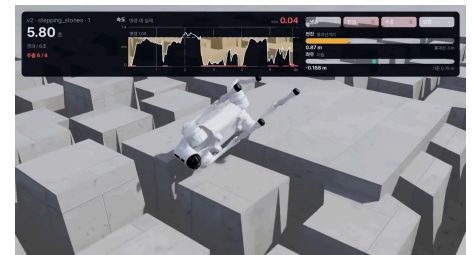


100 판 성공률 0 % · 이 컷은 6.00 초 한 판

V08 · 영상 보기 · 대표 장면 2.4초 / 6초

지형을 마주한 보행 장면. 이 한 판의 모습과 캡션의 100회 평가 성공률을 구분한다.

foothold-v2



100 판 성공률 24 % · 이 컷은 6.00 초 한 판

V09 · 영상 보기 · 대표 장면 5.8초 / 6초

디딤돌 사이에서 자세가 무너지는 장면. 이 사례가 전체 평가 성공률 24%를 대표하지는 않는다.

여덟 중에서 큰 몫은 **NVIDIA** -> **foothold-v1** 이다 (43.88 -> 86.71). v2 가 더한 것은 **1.75 %p** 다 (86.71 -> 88.46 · 2 절 표). 위 세 화면이 그것을 그대로 보인다 · pit 은 v1 이 이미 풀었고, v2 가 눈에 보이게 더한 칸은 **floating_ring** 1.5(+12 %p)와 **stepping_stones** 1.0(+24 %p) 둘이다.

3. 성공률을 구성하는 네 조건

성공률은 넷의 AND 다. 그 넷을 펴 보면 **NVIDIA** 가 왜 **41 %** 인지가 달라진다.

성분 (d0.5 · 48칸)	NVIDIA	v1	v2	무엇을 재나
survival_rate	87.3	95.0	96.6	안 넘어졌나
progress_success_rate	66.8	92.4	95.3	앞으로 갔나
tracking_success_rate	44.5	90.7	95.4	명령 속도를 따라갔나
direction_success_rate	63.8	90.7	95.3	진로를 지켰나

NVIDIA 의 **41 %** 는 「넘어져서」가 아니다. 넘어지지 않는 비율은 87.3 이다. **명령 속도를 못 따라간다** (44.5). 험지에서 느려지고, 그러면 통과 판정에 걸린다.

이것을 안 펴면 「**NVIDIA** 는 험지에서 넘어진다」고 잘못 읽는다.

속도별

판	0.5 m/s	1.0 m/s	1.5 m/s
NVIDIA	55.1	57.4	10.9
foothold-v1	90.8	90.4	85.4
v2	93.7	95.2	93.6

1.5 m/s 에서 **갈린다**. 다만 1.5 m/s 는 **NVIDIA** 의 학습 명령 범위 **lin_vel_x** (-1, 1) 밖이다. 그 사실을 같이 적어야 공정하다.

4. 지형별 성능

4-0. 지형 열여섯 종을 한 표에서 · 컷은 갤러리로

여기서 숫자를 보고 궁금한 칸만 눌러 갑니다. 칸은 **NVIDIA** / **v1** / ****v2**** 순서이고 **지형·속도마다 100 판** 이다 (난이도 0.5). 「컷」을 누르면 갤러리의 그 칸으로 가서 세 판을 나란히 재생할 수 있다.

지형	묶음	0.5 m/s	1.0 m/s	1.5 m/s	컷
discrete_obstacles	어느 판도 학습 안 함	94 / 100 / 100	94 / 100 / 100	0 / 100 / 100	갤러리
floating_ring	어느 판도 학습 안 함	1 / 95 / 100	0 / 99 / 100	0 / 87 / 99	갤러리
pit	어느 판도 학습 안 함	7 / 100 / 100	2 / 100 / 100	0 / 100 / 100	갤러리
repeated_boxes	어느 판도 학습 안 함	100 / 100 / 100	100 / 100 / 100	1 / 100 / 100	갤러리
repeated_cylinders	어느 판도 학습 안 함	99 / 100 / 100	100 / 100 / 100	7 / 100 / 100	갤러리
star	어느 판도 학습 안 함	86 / 100 / 100	100 / 100 / 100	62 / 100 / 100	갤러리
stepping_stones	어느 판도 학습 안 함	0 / 0 / 0	0 / 0 / 24	0 / 0 / 0	갤러리
wave	어느 판도 학습 안 함	100 / 100 / 100	100 / 100 / 100	0 / 100 / 100	갤러리

지형	류음	0.5 m/s	1.0 m/s	1.5 m/s	컷
boxes	셋 다 학습	76 / 99 / 100	34 / 100 / 100	0 / 100 / 100	갤러리
gap	친척 학습	0 / 90 / 100	1 / 100 / 100	0 / 68 / 99	갤러리
hf_pyramid_slope	셋 다 학습	100 / 100 / 100	100 / 100 / 100	0 / 100 / 100	갤러리
hf_pyramid_slope_inv	셋 다 학습	100 / 100 / 100	100 / 100 / 100	24 / 100 / 100	갤러리
pyramid_stairs	셋 다 학습	10 / 100 / 100	100 / 100 / 100	81 / 100 / 100	갤러리
pyramid_stairs_inv	셋 다 학습	24 / 100 / 100	0 / 100 / 100	0 / 100 / 100	갤러리
rails	v2 학습	0 / 70 / 99	8 / 48 / 100	0 / 21 / 100	갤러리
random_rough	셋 다 학습	85 / 99 / 100	80 / 100 / 100	0 / 90 / 100	갤러리

표가 말하는 것 셋.

1. NVIDIA는 1.5 m/s에서 종합 성공률이 크게 낮아진다. 낙상 여부는 생존 지표와 구분해 해석한다.
discrete_obstacles 94 -> 0 · repeated_boxes 100 -> 1 · wave 100 -> 0 · repeated_cylinders 100 -> 7. 느린 데서 되던 것이 빠르면 안 된다.
2. 48 칸 모두 v2 >= NVIDIA 다 확인됨. 내려간 칸이 0 개 다.
3. v2 가 100 % 가 아닌 칸은 6 / 48 뿐이다 (floating_ring 1.5 m/s 99 % · gap 1.5 m/s 99 % · rails 0.5 m/s 99 % · stepping_stones 0.5 m/s 0 % · stepping_stones 1.0 m/s 24 % · stepping_stones 1.5 m/s 0 %).

지형 하나하나에서 무엇이 달라졌나

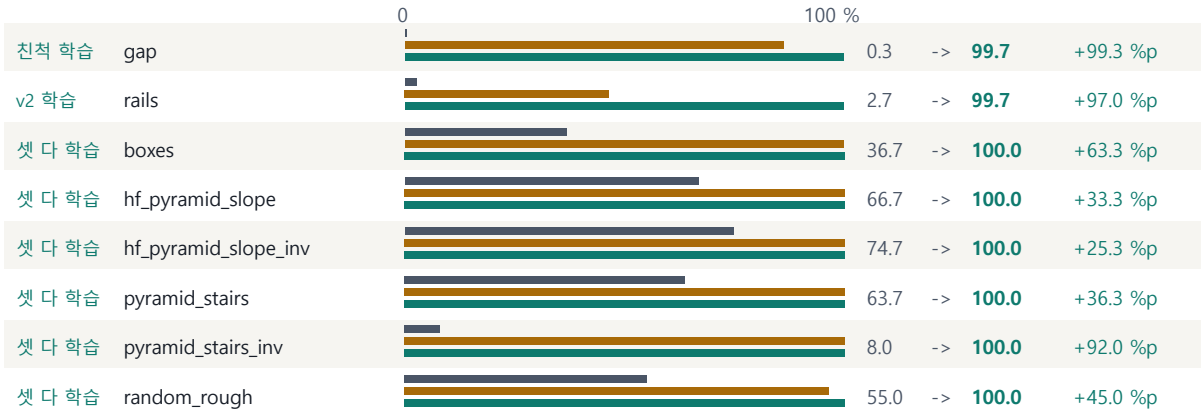
난이도 0.5 · 속도 셋 평균 · 지형마다 300판



지형 하나하나에서 무엇이 달라졌나

난이도 0.5 · 속도 셋 평균 · 지형마다 300판

■ NVIDIA ■ foothold-v1 ■ v2 (v2g2 @3000)



미경험 여덟 중 stepping_stones 하나만 8.0 % 다 · 나머지 일곱은 99.7 % 이상

그림 3 · 지형 열여섯 종의 성공률. 왼쪽 고리표가 그 지형이 각 판에게 학습이었는지 미경험이었는지를 밝힙니다

고리표	지형	NVIDIA	v1	v2	v2 - NVIDIA
미경험	stepping_stones	0.0	0.0	8.0	+8.0
미경험	floating_ring	0.3	93.7	99.7	+99.3
미경험	discrete_obstacles	62.7	100.0	100.0	+37.3
미경험	pit	3.0	100.0	100.0	+97.0
미경험	repeated_boxes	67.0	100.0	100.0	+33.0
미경험	repeated_cylinders	68.7	100.0	100.0	+31.3
미경험	star	82.7	100.0	100.0	+17.3
미경험	wave	66.7	100.0	100.0	+33.3
친척 학습	gap	0.3	86.0	99.7	+99.3
v2 학습	rails	2.7	46.3	99.7	+97.0
셋 다 학습	boxes	36.7	99.7	100.0	+63.3
셋 다 학습	hf_pyramid_slope	66.7	100.0	100.0	+33.3
셋 다 학습	hf_pyramid_slope_inv	74.7	100.0	100.0	+25.3
셋 다 학습	pyramid_stairs	63.7	100.0	100.0	+36.3
셋 다 학습	pyramid_stairs_inv	8.0	100.0	100.0	+92.0
셋 다 학습	random_rough	55.0	96.3	100.0	+45.0

rails 의 46.3 -> 99.7 은 일반화가 아니라 학습이다. v1 은 안 배웠고 v2 는 배웠다. 그래서 이 줄을 일반화 근거로 인용하지 않는다.

pyramid_stairs_inv 에서 NVIDIA 가 8.0 인 것은 그것이 NVIDIA 의 학습 지형인데도 그렇다. 1500 회 배포본의 한계이지 우리 평가의 결함이 아니다 (같은 하네스에서 v1 과 v2 가 100.0 을 낸다).

5. 난이도 변화에 따른 성능

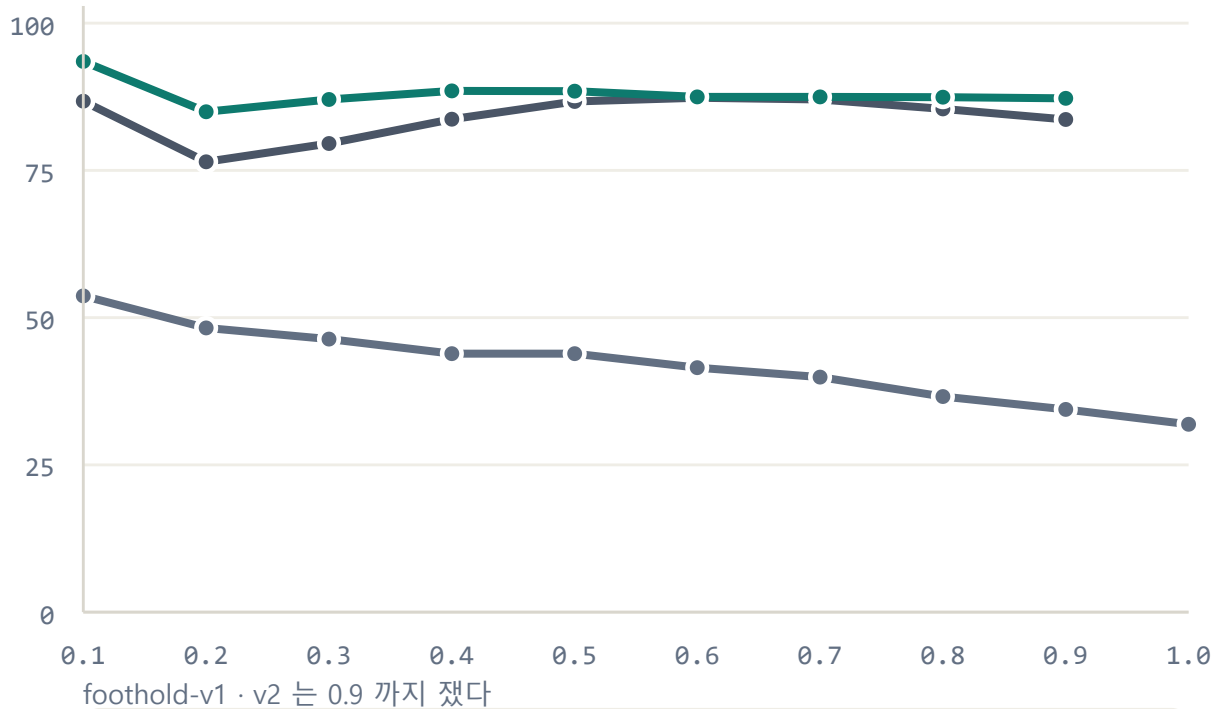
어느 지형을 «미경험» 이라 부르냐에 따라 이야기가 뒤집힌다

세로는 성공률 % · 가로는 지형 난이도 · 두 묶음의 다른 점은 rails 와 gap 둘뿐이다

■ NVIDIA ■ foothold-v1 ■ v2 (v2g2 @3000)

어느 판도 학습 안 한 8종

rails 와 gap 을 뺀 것



v2 - v1 은 +6.8 에서 시작해 +3.6 로 끝난다 (줄어진다)

여덟 점의 폭 +0.2 ~ +8.5 %p

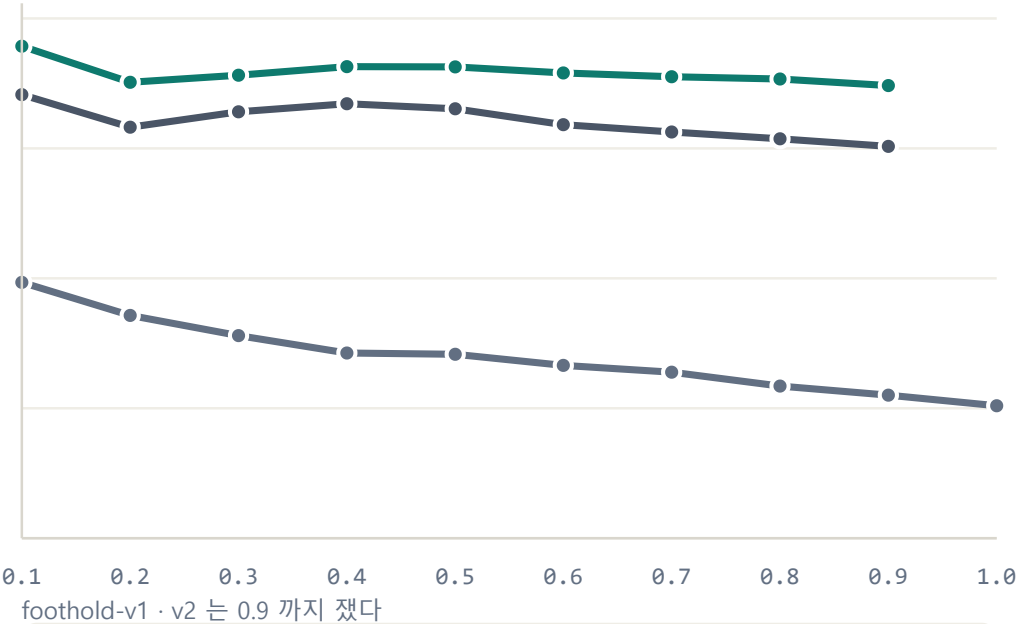
어느 지형을 «미경험» 이라 부르냐에 따라 이야기가 뒤집힌다

세로는 성공률 % · 가로는 지형 난이도 · 두 묶음의 다른 점은 rails 와 gap 둘뿐이다

■ NVIDIA ■ foothold-v1 ■ v2 (v2g2 @3000)

미경험 10종 (rails · gap 포함)

그 둘은 v2 가 학습한 지형이다



v2 - v1 은 +9.3 에서 시작해 +11.7 로 끝난다 (벌어진다)
여덟 점의 폭 +7.1 ~ +11.7 %p

시드만 바뀌도 1.25 %p 가 흔들린다 (멈출 기준 문서). 왼쪽 판의 중간 난이도 차이는 그 폭 안이라 구별되지 않는다

그림 4 · 두 묶음을 나란히. 어느 지형을 미경험이라고 부르냐에 따라 v1 과의 차이가 벌어지기도 하고 좁아지기도 합니다

어느 판도 학습 안 한 여덟 종

난이도	NVIDIA	v1	v2	v2 - NVIDIA
0.1	53.7	86.8	93.5	+39.8
0.2	48.2	76.5	85.0	+36.7
0.3	46.4	79.6	87.0	+40.7
0.4	43.9	83.7	88.5	+44.6
0.5	43.9	86.7	88.5	+44.6
0.6	41.5	87.3	87.5	+46.0
0.7	39.9	87.0	87.5	+47.6
0.8	36.6	85.4	87.4	+50.8
0.9	34.4	83.6	87.2	+52.8
1.0	31.9	없음	없음	

NVIDIA 는 53.7 에서 31.9 로 내려가고 v2 는 93.5 에서 87.2 로 버틴다. 차이가 +39.8 에서 +52.8 로 벌어진다. 그런데 v1 과의 차이는 반대로 간다. +6.8 에서 시작해 중간 난이도에서 +0.2 까지 좁아졌다가 +3.6 으로 끝난다. 중간 난이도의 여유는 시드 폭 1.25 %p 안이라 구별되지 않는다 (8 절).

여기서 조심할 것

unseen10 열 중으로 그리면 v1 과의 차이가 +9.3 에서 +11.7 로 벌어진다. 두 묶음이 다른 것은 rails 와 gap 뿐이고 그 둘이 v2 가 학습한 지형이다. 어려워질수록 학습 지형의 우위가 커지는 것은 일반화가 아니다. 그래서 이 보고서는 여덟 종 쪽을 정본으로 쓴다.

지형마다 난이도를 다르게 탄다

지형	v2 · d0.1	v2 · d0.9
stepping_stones	100.0	0.0
floating_ring	48.0	100.0
pit	100.0	98.0
나머지 다섯	100.0	100.0

floating_ring 은 어려울수록 «쉬워진다». 난이도가 고리 치수를 바꾸는 방향 때문으로 보이나 아직 안 뜯어봤다 미확인.

6. 연구 과정 · 문제, 가설, 실험과 검증

여기서부터가 이 판이 어떻게 나왔는지다. 숫자만 보면 어느 설정을 왜 건드렸는지 안 보인다.

한 칸씩 바꿔서 여기까지 왔다

가로 막대는 난이도 0.5 의 48 칸 평균 % · 오른쪽은 그 판에서 «무엇을 바꿨나»
E · F · G 는 D 에서 «두 칸» 씩 바뀌었다 (lin_vel_x 하한 0.4 가 공통)

NVIDIA		41.17	출발점 · 학습 6종
A · B		88.88	학습에 틈을 넣는다
D		87.19	명령 다섯을 되돌린다
E		88.23	바라보는 비율 0.75
F		89.71	노출을 지형으로 (omni_gap)
G		82.83	heading 을 끈다
H		86.33	F 의 지형 + D 의 명령
v2a		93.00	rails 0.10 추가
v2b		93.44	rel_standing_envs 0.1
v2g2		94.19	feet_air_time 0.1

G 가 가장 낮다 (82.83). heading 을 아예 끄면 무너진다 · F 가 가장 높다 (89.71). 노출을 지형으로 옮긴 쪽이 이겼다

그림 5 · A 에서 v2g2 까지. 판마다 무엇을 한 칸 바꿨고 그때 48칸 평균이 얼마였나

G 가 가장 낮고 (82.83) F 가 가장 높다 (89.71). heading 을 아예 끄면 무너지고, 노출을 지형으로 옮긴 쪽이 이겼다. 아래에서 하나씩 본다.

6-0. 출발점 · NVIDIA Go2 rough

학습 지형 6종 pyramid_stairs · pyramid_stairs_inv · boxes · random_rough
hf_pyramid_slope · hf_pyramid_slope_inv
명령 lin_vel_x (-1, 1) · lin_vel_y (-1, 1) · ang_vel_z (-1, 1)
학습량 1500 회 · learning_rate 1.0e-3 · 처음부터
신경망 actor · critic 각 [512, 256, 128] · elu · 관측 235 차원

6-1. A · B → foothold-v1 · 왜 하필 gap 이 첫 타자였나

문제 파악. 높이 스캔 한 줄기가 내는 값이 이렇다.

평지 -0.10 바닥이 발밑에 있다
턱이 솟음 -0.30 값이 더 음수로
바닥이 꺼짐 +1 광선이 아무것도 못 맞힌다

기존 학습 6종에는 솟은 것만 있고 꺼진 것이 없다. 그래서 구멍을 어떻게 처리하는지 코드로 다룬 적이 한 번도 없었다. 양수 쪽 값을 정책이 학습에서 만난 적이 없다는 뜻이다.

가설. 기대 효과가 가장 클 자리가 여기다. 학습 지형에 틈을 넣으면 건넌다.

실험. 선행 실험에서 개발한 gap 레시피를 재현한 것이 A 이고, 학습 틈 폭을 평가 기준과 같게 넓힌 것이 B 다. 둘의 차이는 gap_width_range 한 줄뿐이다.

검증. B 가 gap 86 % 까지 올라 foothold-v1 으로 배포됐다.

그런데 대가를 치렀다.

명령	NVIDIA	foothold-v1
lin_vel_x	(-1, 1)	(0.5, 1.5)
lin_vel_y	(-1, 1)	(0, 0)
ang_vel_z	(-1, 1)	(0, 0)
rel_standing_envs	0.02	0.0

전진 전용이다. 멈출 줄도 돌 줄도 모른다.

6-2. 「전진 성공률만 보고 있다」가 왜 문제인가

(1) 숫자가 능력보다 커 보인다. foothold-v1 의 gap 86 % 는 항상 틈을 향해 서 있던 조건에서 나온 값이다. 명령이 전진뿐이라 로봇이 딴 데를 볼 수가 없었다. 그 조건의 주행 성적이지 일반화 능력의 크기가 아니다.

(2) 못하는 것이 안 보인다. 능력 열세 가지 중 재고 있던 것이 셋뿐이었다. 못 멈추고 못 도는데 성적표에 그 칸이 없었다.

(3) 쓸 수가 없다. 사람 앞에서 못 서는 로봇은 험지를 아무리 잘 건너도 내보낼 수 없다.

6-3. D · 명령 능력을 되찾는다

가설. 닫았던 명령을 열면 되찾는다.

실험. foothold-v1 설정에서 필드 다섯을 되돌렸다.

```
lin_vel_x      (0.5, 1.5) -> (0.0, 1.5)
ang_vel_z      (0.0, 0.0) -> (-1.0, 1.0)
heading_command False    -> True
rel_heading_envs 0.0      -> 1.0
rel_standing_envs 0.0     -> 0.02
```

그리고 **축 2 하네스를 새로 만들었다**. 평지에서 명령에만 반응시키는 프로브다. 지형을 섞으면 정지 실패가 명령 탓인지 지형 탓인지 안 갈린다.

여기서 소스를 읽어 한 쌍을 찾았다. **heading_command** 와 **ang_vel_z** 는 한 쌍이다 (velocity_command.py:155).

```
clip(stiffness × heading_error, ang_vel_z[0], ang_vel_z[1])
```

ang_vel_z 가 (0,0) 이면 heading 을 커도 **회전이 죽는다**.

검증. 멈추고 도는 것은 배웠는데 **gap 을 잃었다** (86 -> 33.3).

6-4. gap 하락의 원인 · 이것은 문제 파악이다

처음 가설 「학습량이 모자라다」 -> 버렸다.

문제 파악. 노출 구조다. 명령을 넓히면 로봇이 틈을 향해 서지 않는다. 그러면 커리큘럼이 틈 난이도를 올려 주지 않고, 안 만난 것은 못 배운다.

terrain_levels_vel 이 직선 거리로 승급을 정하는데 (curriculum.py:47-51 · size/2 = 4 m), 로봇이 옆으로 돌면 그 거리가 안 늘어난다.

학습량 대조도 실제 노출 계측도 아직 안 했다 미확인.

같은 gap 을 세 판이 어떻게 건너나

볼 것: 같은 gap 을 세 판이 어떻게 건너는가. 명령을 열었더니 (foothold-v1 -> D) 틈을 잃었고, omni_gap 을 넣었더니 (D -> v2) 되찾았다. gap 성공률이 **86.0 -> 33.3 -> 99.7** 이다.

그 수는 난이도 0.5 의 «세 속도 평균» 이다. 세 컷은 전부 난이도 0.5 · **1.0 m/s** 한 칸이고, 그 칸만 보면 foothold-v1 도 v2 도 100 % 다 (sweep_long.csv 실측). 곧 이 컷들은 하락을 숫자로 보이는 자리가 아니라 건너는 «방식» 이 어떻게 달라졌는지를 보이는 자리다.

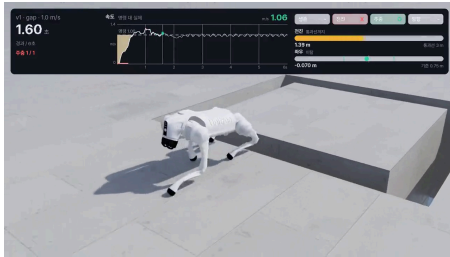
foothold-v1 의 gap 세 칸을 펴 보면 **0.5 m/s 90 % · 1.0 m/s 100 % · 1.5 m/s 68 %** 이고 평균이 86.0 이다. 곧 평균을 끌어내린 것은 **1.5 m/s** 다. v2 는 같은 세 칸이 100 · 100 · 99 다.

D 의 컷을 «못 건너는» 판으로 같았다. 앞 판은 세 컷이 다 건너는 판이라 하락이 장면에 안 보였다 (전진 5.68 · 5.60 · 5.87 m). 시드를 훑어 다시 찍었고 **시드 46** 에서 전진 **0.99 m** 로 떨어지는 판을 잡았다.

그 판은 이 속도에서 소수 쪽이다 확인됨. 같은 조건에서 시드 다섯을 훑으니 42 · 43 · 44 · 45 는 5.60 · 5.50 · 5.40 · 5.41 m 로 건넌고 46 만 0.99 m 였다. **33.3 %** 는 세 속도 평균이고 **1.0 m/s** 칸의 값이 아니다. 그래서 이 컷은 「D 는 대개 못 건넌다」가 아니라 「D 는 떨어질 때가 있는데 그 모습이 이렇다」로 읽어야 한다.

지형은 셋 다 평가 지형 gap 이다. omni_gap 컷이 아니다. omni_gap 은 v2 의 «학습» 지형이라 평가 목록에 없다. 생김새는 **6-6 의 비교 컷**에서 볼 수 있다.

foothold-v1



건넌다 · 전진 5.68 m · gap 세 속도 평균 86.0 %
V10 · 영상 보기 · 대표 장면 1.6초 / 6초

틈을 건너는 중간 동작 캡션의 전진 거리는 영상 전체의 최종값이며, 대표 장면 시각의 HUD 값과 다를 수 있다.

D (명령을 연 판)



못 건넌다 · 전진 0.99 m (시드 46) · gap 세 속도 평균 33.3 %

V11 · 영상 보기 · 대표 장면 2.4초 / 6초

틈 앞에서 몸이 걸려 진행하지 못하는 장면 캡션의 전진 거리는 영상 전체의 최종값이며, 대표 장면 시각의 HUD 값과 다를 수 있다.

foothold-v2



건넌다 · 전진 5.87 m · gap 세 속도 평균 99.7 %
V12 · 영상 보기 · 대표 장면 1.6초 / 6초

동일 시점에서 틈을 건너는 동작 캡션의 전진 거리는 영상 전체의 최종값이며, 대표 장면 시각의 HUD 값과 다를 수 있다.

그런데 v2 가 학습한 것은 gap 이 아니라 omni_gap 이다. 둘은 함수가 다르다. 지형 자체의 생김새는 그림 1 에서 볼 수 있다.

무엇	어디	생김새
평가의 gap	그림 1 · gap 타일	사각 구멍이 바닥에 뚫려 있다 (MeshGapTerrainCfg)
v1 이 학습한 forward_gap	6-6 의 컷 (평가 목록에는 없다)	스폰 +x 쪽에만 틈이 있다
v2 가 학습한 omni_gap	6-6 의 컷 (평가 목록에는 없다)	스폰을 둘러싼 고리다 (mode=ring)
고리의 눈으로 보는 것	그림 1 · floating_ring 타일	사각 테두리가 스폰을 감싼 모양

forward_gap 은 앞으로만 가면 되는 틈이라 명령을 열면 옆·뒤에서 틈을 만난다. 그것이 D 에서 33.3 % 로 떨어진 까닭이고, omni_gap 으로 바꾼 것이 99.7 % 로 되돌린 까닭이다.

6-5. E · F · G · 설정 하나가 두 일을 한다

문제 파악. heading 하나가 두 일을 한다.

heading 이 하는 일	무엇을 정하나
노출	로봇이 틈을 향해 서는가
방향 유지 압력	진로를 얼마나 지키는가

범위를 어디에 뒀도 둘이 같이 움직인다. 어느 쪽이 효과인지 못 가른다.

실험. 셋을 걸었다.

```

E = D + lin_vel_x 하한 0.4 + rel_heading 0.75
F = D + lin_vel_x 하한 0.4 + 지형을 omni_gap 으로 <- 노출을 지형으로 옮긴다
G = D + lin_vel_x 하한 0.4 + heading 곱 <- 방향 유지 압력만 댄다
H = F 의 지형 + D 의 명령 (하한 0.0 으로 되돌림)
  
```

검증 (난이도 0.5 · 48칸)

판	48칸	미경험 10종	학습 6종	8종	gap
D	87.19	80.40	98.50	85.71	33.3
E	88.23	82.17	98.33	86.62	99.7
F	89.71	83.93	99.33	84.25	99.0

판	48칸	미경험 10종	학습 6종	8종	gap
G	82.83	73.10	99.06	76.46	80.7
H	86.33	79.20	98.22	85.71	73.7

1. E 가 gap 을 되찾았다 (33.3 -> 99.7)
 2. F 가 48칸 89.71 로 가장 높다. 노출을 지형으로 옮긴 쪽이 이겼다
 3. G 가 82.83 으로 가장 낮다. heading 을 아예 고니 8종이 76.46 으로 무너졌다. 방향 유지 압력을 다 떼면 안 된다
- 다만 셋 다 D 에서 «두 칸» 씩 바뀌었다. lin_vel_x 하한 0.4 가 공통이라 단일 원인 귀속이 안 된다 미확인.

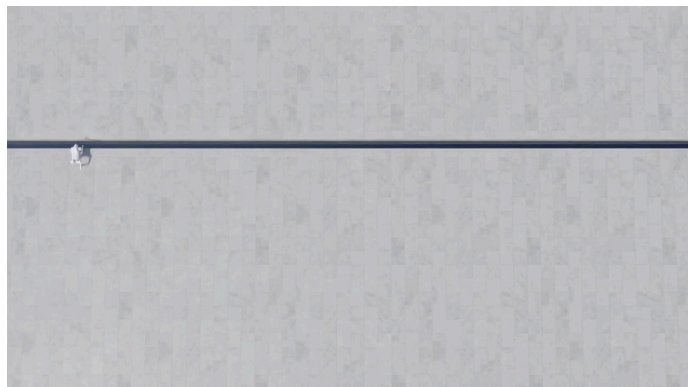
6-6. 왜 omni_gap 이었나

틈을 «사방에» 두면 로봇이 어디를 보든 틈을 만난다. 명령을 열어 둔 채로도 노출이 유지된다. 명령으로 노출을 만들려다 실패했으니 지형이 대신 만들게 한 것이다.

```
forward_gap  +x 에만 있는 한 줄 틈.  로봇이 돌면 안 만난다
omni_gap     스폰을 둘러싼 고리.    어디로 돌든 만난다 (mode: ring · ring_radius 2.5)
```

볼 것: 학습 지형이 어떻게 바뀌었는가. 둘 다 바로 위에서 본 것이고 3x3 타일이다. 왼쪽은 직선 도랑 하나가 화면을 가로지르고, 오른쪽은 고리 아홉 개가 타일마다 스폰을 감싼다. 로봇이 크기 자를 겸한다.

forward_gap · v1 이 학습



스폰 앞을 가로지르는 직선 도랑. 돌아서면 안 만난다

V13 · 영상 보기 · 대표 장면 1.5초 / 3초

직선 도랑과 시작 위치의 관계

omni_gap · v2 가 학습



스폰을 둘러싼 고리 (mode=ring · 반지름 2.5 m). 어디로 돌든 만난다

V14 · 영상 보기 · 대표 장면 1.5초 / 3초

시작 위치를 둘러싼 고리형 틈

이 두 컷의 출처. 학습 cfg 를 Isaac Lab 설치본에서 가져와 찍었다 (isaacsim_tasks...config.go2.gap_training.gap_e_env_cfg · ...gap_h_env_cfg). 저장소 사본 sim/policy/gap_e_env_cfg.py 로는 못 찍는다. 사슬이 저장소 안에서 닫히지 않기 때문이다 확인됨.

그 까닭이 이 판에서 한 고리 뒤로 옮겨 갔다. 처음에는 gap_wide_env_cfg 가 저장소에 아예 없었고, PR #461 이 머지되어 (472f0432) 이제 있다. 그런데 그 다음 고리인 gap_env_cfg 13 행이 ..rough_env_cfg 를 부르고, 그 파일은 Isaac Lab 것이라 저장소에 없다 (sim/rough_env_cfg.py 도 sim/policy/rough_env_cfg.py 도 없다 확인됨). 그래서 컷은 여전히 설치본으로 찍는다.

설치본은 학습을 실제로 돌린 파일이고, 저장소 사본 셋(gap_e · gap_h · omni_gap_terrain)과 줄끝만 빼고 한 글자도 같다 확인됨.

저 검은 띠가 타일 경계가 아니라 «틈» 인 것은 omni_gap 쪽에 직선 띠가 없는 것으로 갈랐다. 같은 격자 · 같은 테두리인데 한쪽에만 난다. 2x2 로 먼저 찍었더니 도랑이 화면 테두리에 잘려서 3x3 으로 다시 찍었다.

6-7. 축을 둘로 못 박는다

	무엇	어떻게
축 1	험지 주행	16 지형 × 3 속도 × 100 판 · d0.5 · 시드 42 · foothold-v1 대비 Wilson 95 %
축 2	명령 응답	평지 · 프로브 아홉 칸

둘의 AND 이고 체크포인트 넷(1500 · 2000 · 2500 · 3000)에서 다 통과해야 한다.

6-8. v2a · 왜 rails 를 넣었나

문제 파악. omni_gap 으로 노출은 풀었는데 학습 지형이 여전히 **평면 위의 요철** 중심이다. 솟은 것을 넘는 동작이 학습에 없다.

근거가 된 경험. A 에서 gap 을 0.1 비율로 넣었을 때 얻은 것이 있었다. **지형 한 종을 0.1 로 더하는 것만으로 그 종류의 능력이 붙는다**는 것이다.

가설. 같은 관점으로 rails 를 0.1 로 더하면 솟은 것을 넘는 능력이 붙는다.

실험. 학습 지형을 8종으로. 명령은 D 의 복원을 유지하되 lin_vel_x 하한을 0.4 로 다시 좁혔다. lin_vel_y 는 **잠근 채** (0,0) 뒀다.

```
pyramid_stairs 0.15 · pyramid_stairs_inv 0.15 · boxes 0.15 · random_rough 0.15
hf_pyramid_slope 0.10 · hf_pyramid_slope_inv 0.10 · omni_gap 0.10 · rails 0.10
```

검증

	iter 1500	iter 2000	iter 2500	iter 3000
48칸 평균	90.79	92.62	91.96	93.00
축 1 진짜 하락	2	1	2	0
축 2 회전 낙상	0.9219	0.1562	0.2812	0.3438

축 1 은 올랐고 축 2 회전이 무너졌다. iter1500 에서 64 판 중 59 판이 넘어진다.

그리고 스스로 한 칸을 깎았다. rails 가 학습과 평가에 둘 다 있으므로 「미경험 험지」 주장에서 뺀다.

6-9. v2b · 왜 «속도 하한» 이 아니라 rel_standing_envs 였나

문제 파악. 회전 낙상. 그리고 v2a 는 lin_vel_x 하한이 0.4 라 「**멈춤**」을 명령으로 배울 기회가 거의 없다.

왜 속도 하한을 0 으로 안 내렸나. 원문을 열어서 확인했다 (velocity_command.py:141,162-163).

```
self.is_standing_env[env_ids] = r.uniform(0.0, 1.0) <= self.cfg.rel_standing_envs
standing_env_ids = self.is_standing_env.nonzero(as_tuple=False).flatten()
self.vel_command_b[standing_env_ids, :] = 0.0
```

rel_standing_envs 는 환경의 «일부» 를 골라 명령을 통째로 0 으로 만든다. 안 고른 환경은 lin_vel_x 에서 뽑은 속도를 그대로 받는다.

```
속도 하한을 0 으로      모든 환경이 느려질 수 있다 -> 직선 거리가 안 늘어난다
                        -> 커리큘럼이 안 올린다 -> 6-4 의 노출 붕괴가 다시 온다
rel_standing_envs 올림  일부만 완전 정지 · 나머지는 0.4 ~ 1.5 m/s 로 그대로 간다
```

정지를 배우는 자리와 험지를 배우는 자리를 나눈 것이다.

실험. 한 칸만 바꿨다. `rel_standing_envs` 0.02 -> **0.1**.

검증

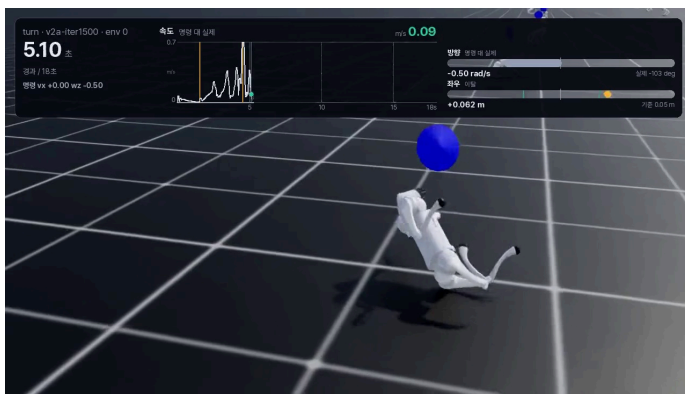
	iter 1500	iter 2000	iter 2500	iter 3000
48칸 평균	93.08	93.31	93.60	93.44
축 1 진짜 하락	0	0	0	0
축 2 회전 낙상	0.0625	0	0.2969	0.0469

한 칸이 두 축을 같이 올렸다. 회전 낙상 0.9219 -> 0.0625.

그 한 칸을 장면으로

볼 것: `rel_standing_envs` 한 칸을 0 에서 0.1 로 올린 것이 회전을 살린다.

v2a iter1500

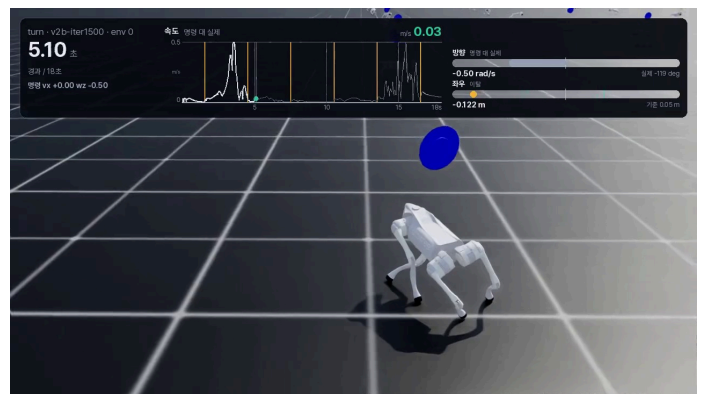


`rel_standing_envs` 0.02 · 64 판 중 59 판이 옆으로 넘어진다

V15 · 영상 보기 · 대표 장면 5.1초 / 18초

회전 중 몸이 옆으로 넘어지는 장면. 리셋 이전

v2b iter1500



`rel_standing_envs` **0.1** · 네 발로 버티며 돈다 · 64 판 중 4 판

V16 · 영상 보기 · 대표 장면 5.1초 / 18초

비교 모델과 동일 시점에서 지지한 회전 자세

`rel_standing_envs` 한 칸만 다른 두 판이다. 숫자로는 0.9219 대 0.0625 인데 화면에서는 옆으로 자빠지는 것과 네 발로 버티며 도는 것으로 갈린다.

판정에는 이 컷의 수를 쓰지 않는다. `--scenario turn` 만 따로 돌리면 `stop -> ramp -> turn -> hold` 를 이어 돌 때와 난수 소비가 달라 0.8438 대 0.0312 가 나온다. 판정 값은 네 시나리오를 이어 둔 것이고 이 컷은 장면으로 보여 주는 용도다.

6-10. v2b 의 자식들 · `feet_air_time` 을 왜 건드렸나

문제 파악. 우리 기획서가 이미 적어 둔 것이다.

현재 정책은 관절이 63.7 cm 까지 발을 들 수 있는데 3.25 cm 만 든다. `feet_air_time` 이 높이가 아니라 시간을 점수로 주고, 발 높이 향이 없다. 여기가 실패 5종의 공통 뿌리다

그리고 rails 는 평지 위로 솟은 사각 테두리 두 겹이다. 넘으려면 발을 들어야 한다.

가설. 발 드는 행동에 보상을 조금 주면 넘는다.

실험. `feet_air_time` 이 발 관련 보상항의 전부다. 한 칸만 바꿔 둘을 걸었다.

기준 (v2a · v2b · v2n · v2s)	0.01	
v2g	1.0	(100 배)
v2g2	0.1	(10 배)

검증 (네 체크포인트 · 48칸 평균)

판	바꾼 한 칸	iter 1500	iter 2000	iter 2500	iter 3000
v2b-r	없음 (기준 재현)	92.06	93.08	93.12	92.79
v2n-noise02	height scan 잡음 $\pm 0.10 \rightarrow \pm 0.02$	91.88	92.90	93.06	93.44
v2s-stones10	stepping_stones 10 % 추가	92.77	91.92	93.38	91.79
v2g-feetair1	feet_air_time 1.0	91.29	92.71	93.23	93.08
v2g2-feetair01	feet_air_time 0.1	92.94	93.56	93.19	94.19
v2sg	위 둘 결합	91.25	90.25	92.27	91.69

0.1 이 최고이고 1.0 은 오히려 낮다. 「조금」이 맞았다.

그리고 stepping_stones 를 학습에 넣은 v2s 가 오히려 91.79 로 낮다. 그 지형을 학습에 넣는 것이 답이 아니었다.

6-11. 시드 재현 · v2g3 · v2g4

같은 레시피를 시드만 바꿔 둘 더 걸었다.

판	시드	iter 1500	iter 2000	iter 2500	iter 3000
v2g2-feetair01	42	92.94	93.56	93.19	94.19
v2g3-feetair01-s43	43	91.83	92.83	93.27	92.94
v2g4-feetair01-s44	44	92.48	93.08	93.44	93.52

이 셋의 폭이 멈출 기준의 바닥선이 된다 (8 절).

6-12. 학습 중단 기록 · std 와 NaN

앞 판에서는 이 절을 접어 두었다 (<details>). 펼치지 않으면 안 보이므로 접어 둔 것은 안 남긴 것과 화면에서 같았다. 검토를 거쳐 본문에 공개했다.

볼 것: 열한 판이 목표 3000 회를 못 채우고 죽었다. 막대가 마지막 저장 번호다. 원인은 아래에서 하나씩 가른다.

시드를 바꾸니 터졌다

RuntimeError: normal expects all elements of std >= 0.0

막대는 목표 3000 회 중 «마지막 저장 번호» 다. 짧으면 도중에 죽은 것이다

실행 이름	시드	feet	std			
v2b-s	43	0.01	scalar		1650	죽었다
v2b-s2	43	0.01	scalar		1650	죽었다
v2b-s3instr	43	0.01	scalar		1650	죽었다
v2b-s3instr (재실행)	43	0.01	scalar		1650	죽었다
v2b-s4paired	43	0.01	scalar		875	죽었다
v2g3-feetair01-s43	43	0.1	scalar		3000	완주
v2r4-base-s44	44	0.01	scalar		2700	죽었다
v2Sc-scalar-s44	44	0.01	scalar		2025	죽었다
v2L-logstd-s44	44	0.01	log		3000	완주
v2LG-log-feet01-s44	44	0.1	log		2900	죽었다
v2g4-feetair01-s44	44	0.1	scalar		3000	완주

시드 43 은 0.01 5 판(이름 4)이 다 죽고 0.1 하나가 살았다 · 시드 44 는 0.01 3 판 중 2 이 죽고 log 하나가 살았다

그런데 log 에 0.1 을 함께 준 판도 2900 에서 죽었다. 매개화가 원인이라는 가설이 반증됐다

표본이 작고 조건이 통제되지 않았다. 단일 원인을 특정하지 못한다. 다만 «모든 판에 똑같이 있던 것» 이 하나 있다

· resume

그림 6 · 학습이 터진 열한 판. 막대는 목표 3000 회 중 마지막 저장 번호입니다

RuntimeError: normal expects all elements of std >= 0.0

시드를 바꾸면 터지고 같은 설정이 다른 시드에서 완주했다.

실패 로그 9 건 · 서로 다른 실행 이름 8 개

둘을 같이 적는 까닭 · v2b-s3instr 이 두 번 걸려 두 번 다 같은 1670 에서 죽었다. 재실행도 실제 실패 1 건이다. 이 수는 독립 설정의 수가 아니다.

내가 세운 가설 「표준편차 매개화가 원인이다. log 로 바꾸면 음수가 안 나온다.」 -> scalar 대 log 2 × 2 를 걸었다.

가설이 틀렸다. log 도 터졌다 (v2LG-log-feet01-s44 · 마지막 완료 출력 번호 2923 · 같은 문구).

체크포인트 2800 · 2850 · 2875 · 2900 을 다 열어 텐서 68 개를 봤는데 비유한수가 하나도 없었다. 최초로 비유한수가 생긴 갱신 번호는 미확인이다.

수학에서 exp() 는 양수지만 float 연산에서는 underflow · overflow · NaN 이 다 가능하고, torch 의 std.min() >= 0 검사는 NaN 에서도 실패한다.

매개화	일어났을 수 있는 것
scalar	std 가 음수가 됐을 수 있다
log	log_std 가 NaN 이 됐을 수 있다

두 칸 다 개별 원인 미확인이다.

라이브러리에 관문이 없다 확인됨. rsl_rl/modules/actor_critic.py 전체에서 clamp · clip · isnan · isfinite 를 찾았다. 하나도 없다.

gradient 자르기는 있는데 NaN 을 막지 않는다. 퍼뜨린다.

```
넣은 gradient      [1.0, 2.0, NaN, 3.0]
clip_grad_norm(..., 1.0)
나온 gradient      [NaN, NaN, NaN, NaN]
```

전체 norm 이 NaN 이면 곱하는 계수가 NaN 이라 전부 NaN 이 된다.

추가 검토에서 확인한 한계는 다음과 같다. 16 판은 요인 설계가 아니고 교란이 섞여 있다. 시드 43 안에서는 feet_air_time 0.01 넷이 다 죽고 0.1 하나가 살았고, 시드 44 안에서는 scalar 0.01 둘이 죽고 log 0.01 이 살았다. 표본이 작고 조건이 통제되지 않아 어느 쪽으로도 말할 수 없다.

그런데 모든 판에 똑같이 있던 것이 하나 있었다. resume 이다.

반복적인 학습 중단을 검토하면서 원인 탐색이 「어느 설정이 중단을 유발하는가」에 한정돼 있었음을 확인했다. 변한 설정만 비교하는 방법으로는 모든 실험에서 변하지 않은 변수의 영향을 찾기 어렵다.

7. 학습 방법과 레시피

배포본 v2g2-feetair01 의 학습 시각 저장본 params/env.yaml 과 params/agent.yaml 에서 읽었다.

시작점 · 맨바닥에서 배운 것이 아니다 확인됨

```
params/agent.yaml    (2026-09-28 · 학습 로그에서 직접 읽음)

resume: true
load_run: nvidia_pretrained_source
load_checkpoint: nvidia_pretrained.pt
```

v2 는 NVIDIA 가중치에서 «이어» 3000 회를 돈 판이다. 그래서 이 보고서의 「NVIDIA 대비」는 서로 다른 두 학습의 비교가 아니라 같은 가중치에서 3000 회를 더 돌린 것과 안 돌린 것의 비교다.

model_0.pt 를 「NVIDIA 원본」이라고 부르지는 않는다. actor 가중치를 직접 맞춰 봤더니 원본과 다르다 (a32b4ad7.. 대 df93a7e1..). 첫 갱신 «뒤» 저장본이다. 원본 자체는 nvidia_pretrained.pt 하나뿐이고, 축 1 · 축 2 의 NVIDIA 열이 그것이다.

학습 지형 여덟 종

pyramid_stairs	0.15	hf_pyramid_slope	0.10
pyramid_stairs_inv	0.15	hf_pyramid_slope_inv	0.10
boxes	0.15	omni_gap	0.10
random_rough	0.15	rails	0.10

명령

설정	값	뜻
lin_vel_x	(0.4, 1.5)	앞으로 가는 속도 명령의 범위
lin_vel_y	(0, 0)	옆으로 가는 명령. 잡가 뒀다
ang_vel_z	(-1.0, 1.0)	제자리에서 도는 속도 명령
heading_command	true	바라볼 방향을 주고 요를 자동으로 맞춘다
rel_heading_envs	1.0	그 방식을 쓰는 환경의 비율
rel_standing_envs	0.1	명령을 통째로 0 으로 두는 환경의 비율
heading_control_stiffness	0.5	방향 오차를 요 명령으로 바꾸는 계수

커리큘럼과 규모

설정	값	뜻
max_init_terrain_level	2	학습 시작 때 열어 두는 난이도 층. 0 이면 전부 가장 쉬운 줄에서 시작한다
num_envs	4096	동시에 돌리는 환경 수
max_iterations	3000	학습 횟수
커리큘럼 규칙	terrain_levels_vel	직선 거리 > size/2 = 4 m 면 올리고, cmd_xy × 길이 × 0.5 보다 짧으면 내린다

보상항 · 무엇을 잘하라고 가르쳤나

feet_air_time 만 기준에서 바꿨다 (0.01 -> 0.1). 나머지는 NVIDIA 기본값 그대로다. 발 관련 보상항은 이것 하나뿐이고, 높이가 아니라 «떠 있던 시간» 을 점수로 준다.

8. 탐색 중단의 기준과 한계

규칙은 두 줄이다.

- 규칙 1 한 칸 바뀌 얻은 것 <= 시드만 바뀌 생기는 흔들림
 -> 정책 탐색을 멈춘다. 더 흔들면 잡음을 재는 것이다
- 규칙 2 시드 흔들림이 판정 폭만큼 크다
 -> 튜닝 말고 «자» 를 고친다. 표본을 늘리거나 값과 구간을 같이 적는다

둘은 다른 상황이다. 규칙 1 은 「정책이 천장에 닿았다」이고 규칙 2 는 「자가 무너서 천장이 어디인지 못 본다」이다.

바닥선을 어떻게 얻나

같은 레시피를 시드만 바꿔 최소 셋 돌리고 그 지표의 폭을 잴다. 그 폭이 그 뒤 모든 비교의 바닥선이다. 그보다 작은 차이는 차이가 아니다.

지표	시드 폭	어떻게 얻었나
축 1 · 48 칸 평균	1.25 %p	시드 42 · 43 · 44 = 94.19 · 92.94 · 93.52
축 2 · 아홉 칸	4 칸 / 9	같은 세 시드 = 8/9 · 4/9 · 7/9

실제로 어디서 멈춰야 했나

v2b 에서 한 칸씩 바꾼 자식 여섯의 폭은 2.50 %p 이고 최고 자식이 기준보다 +1.40 %p 였다.

+1.40 과 1.25 가 거의 같다. 규칙 1 에 걸린다.

구간	얻은 것	시드 폭	판정
v2a -> v2b	축 1 하락 2·1·2·0 -> 0·0·0·0	1.25 %p	잡음보다 큼. 계속할 이유가 있었다
v2b -> v2g2	48 칸 93.44 -> 94.19 (+0.75)	1.25 %p	잡음 안. 멈출 자리였다

v2b 에서 멈춰야 했고 자식 다섯 판과 시드 두 판이 초과였다.

그리고 규칙을 그대로 적용하면 이렇게 된다.

v2g2 가 v2b 보다 낫다고 말할 수 없다. 48 칸 평균이 94.19 대 93.44 로 v2g2 가 높지만 시드 폭 1.25 %p 안에 들어간다.

그러면 왜 v2g2 를 골랐나. 성적이 나아서가 아니라 가장 두껍게 잰 판이기 때문이다. 기준선은 잘하는 판이 아니라 잘 잰 판이어야 한다.

축 2 는 왜 사정이 다른가

축 2 는 규칙 2 에 해당한다. 아홉 칸이 전부 문턱을 넘었나 못 넘었나 둘 중 하나인데, 정책들이 하필 문턱 위아래에 몰려 있다.

v2g2 의 wz +1.00 0.3994 문턱 0.40
 0.0006 차이로 한 칸이 통째로 미달이 된다

표본을 늘려도 참값이 문턱 위에 앉아 있으면 안 걸린다. 64 env 에서 1024 env 로 올려 실제로 확인했더니 미달 칸이 회전 낙상 에서 wz +1.00 으로 옮겼을 뿐 개수는 8/9 그대로였다.

그래서 축 2 에서 할 일은 정책을 흔드는 것이 아니라 자를 고치는 것이다.

무엇	기준이 바뀌나	다시 재야 하나	비용
값과 구간(Wilson)을 같이 적는다	아니오	아니오	0
표본을 64 -> 1024 로	아니오	예	칸당 약 2.5 분
문턱 값 자체를 바꾼다	예	아니오	절차 필요 · 결과를 본 뒤에는 하지 않는다

9. 명령 수행 능력 평가

평지에서 명령에만 반응시키는 프로브다. 지형을 섞으면 정지 실패가 명령 타인지 지형 타인지 안 걸린다.

#	시나리오	지표	문턱	무엇을 묻나
1	stop	fell_ratio	<= 0.03	4 초 전진 뒤 명령 0 을 주면 넘어지지 않고 서나
2	hold	fell_ratio	<= 0.03	20 초 내내 명령 0 이어도 안 넘어지나
3	hold	residual_speed_mps	<= 0.005	선 뒤에 미끄러지지 않나
4	hold	joint_target_delta_tail	<= 0.01	선 뒤에 관절이 떨지 않나
5	turn	fell_ratio	<= 0.10	제자리 회전에서 안 넘어지나
6~9	turn	yaw_follow_ratio (-1.0 · -0.5 · +0.5 · +1.0)	>= 0.40	요 명령 네 크기를 각각 따라 도나

아홉 칸이 된 까닭도 코드에 적혀 있다 (verdict_manifest.py:89-102). 2026-09-23 에 8 에서 9 로 올렸다.

NVIDIA 원본이 9/9 다. 거기서 출발했는데 8/9 를 통과로 부르면 출발점보다 나쁜 것을 배포하면서 성공이라 하는 셈이다.

그때 D 판정이 뒤집혔다 (옛 기준 통과 -> 새 기준 미달).

ramp 는 재지만 판정하지 않는다. AXIS2_THRESHOLDS 에 키가 없다.

9-2. 확장 축 · 판정에 안 쓰고 «기록만» 한다

아홉 칸 말고도 하네스가 들고 있는 시나리오가 더 있다. 판정에 넣지 않는다. 문턱을 결과를 본 뒤에 정하면 그것은 자가 아니다. 대신 자료를 쌓는다.

「돌린 판」은 서로 다른 체크포인트의 수다 (policy_sha256 으로 센다). 이름이 달라도 같은 파일이면 한 판이다
확인됨 (nvidia = nvidia-zero · foothold-v1 = v1 · v2 = v2g2-feetair01-iter3000).

시나리오	무엇을 묻나	하네스	돌린 판
slow010 ~ slow040	아주 느린 명령(0.10 ~ 0.40 m/s)을 따라가나	있음	19
turn_rest	멈춘 채로 도는가	있음	4
turn_rev	반대로 도는가	있음	4
push_robot	외란을 견디나	외란 유지 옵션 있음 · 독립 시나리오 없음	0
험지 회전 · 횡보 · 장거리		없음	0

이 표의 자를 고쳤다 · 결론 둘을 철회한다

앞 판은 residual_speed_mps 를 「낮을수록 좋다」로 읽었다. 그것은 끝 1 초의 평균 속도 다 (sim/eval/command_response_metrics.py 320 행). 명령이 0 인 hold 에서는 낮을수록 좋다. 그런데 slow010 은 0.10 m/s 로 가라고 명령한 것 이다. 0 에 가까우면 「안 움직였다」는 뜻이고 그것은 좋은 것이 아니다.

하네스가 이미 tracked_vx_mps 와 tracking_ratio 를 주고 있었다. 자를 바꿔 재니 네 칸 모두 1 위가 바뀐다 확인됨.

자	slow010	slow020	slow030	slow040
잔류 속도가 가장 낮은 판 (앞 판의 자)	NVIDIA	NVIDIA	NVIDIA	NVIDIA
명령 대비 오차가 가장 작은 판	v2a	v2a	v2b	v2b

철회 1 · 「NVIDIA 가 저속에서 가장 낮다」. NVIDIA 는 저속 명령을 거의 안 따른다 (추종비 0.06 · 0.30 · 0.71 · 0.87). 잔류 속도가 낮은 것은 안 움직였기 때문 이다.

철회 2 · 「저속은 v2 계열이 계속 지고 있는 자리다」. 오차로 재면 네 칸 모두 v2 계열이 1 위다.

살아남는 것. v2g2 가 v2b 보다 저속에서 나쁜 것은 자를 바꿔도 맞다 (오차 0.067 대 0.016 · 0.065 대 0.021 · 0.012 대 0.000 · 0.024 대 0.009). 다만 까닭이 반대 다. 덜 가는 것이 아니라 넘어선다 (추종비 1.67).

저속 명령 · 실제 전진 속도 (m/s · 최대 64개 환경 중 유효 표본 평균 · 명령에 가까울수록 좋다)

판	slow010 명령 0.10	slow020 명령 0.20	slow030 명령 0.30	slow040 명령 0.40
NVIDIA	0.006	0.061	0.213	0.347
foothold-v1	0.092	0.199	0.281	0.371
D	0.063	0.166	0.273	0.374
v2a @3000	0.107	0.199	0.276	0.369
v2b @3000	0.084	0.221	0.300	0.391
v2g2 @3000 (배포본)	0.167	0.265	0.312	0.376

추종비 (실제 / 명령 · 1.00 이 딱 맞는 것 · 1 보다 크면 넘어선 것)

판	slow010	slow020	slow030	slow040
NVIDIA	0.06	0.30	0.71	0.87
foothold-v1	0.92	1.00	0.94	0.93
D	0.63	0.83	0.91	0.93
v2a @3000	1.07	1.00	0.92	0.92
v2b @3000	0.84	1.11	1.00	0.98
v2g2 @3000 (배포본)	1.67	1.33	1.04	0.94

|추종 오차| (m/s · 낮을수록 좋다 · 굵은 것이 그 칸 1 위)

판	slow010	slow020	slow030	slow040
NVIDIA	0.094	0.139	0.087	0.053
foothold-v1	0.008	0.001	0.019	0.029
D	0.037	0.034	0.027	0.026
v2a @3000	0.007	0.001	0.024	0.031
v2b @3000	0.016	0.021	0.000	0.009
v2g2 @3000 (배포본)	0.067	0.065	0.012	0.024

두 모델의 오차 방향이 다르다. 0.10 m/s 명령에서 NVIDIA의 평균 전진 속도는 0.006 m/s로 미달하고, v2g2는 0.167 m/s로 초과한다. v2g2의 학습 명령 하한 0.4 m/s가 저속 추종에 영향을 줬을 가능성은 있지만, 이 비교만으로 원인을 분리해 확인한 것은 아니다.

foothold-v1의 slow010 추종비 0.92는 추종비가 산출된 48개 환경의 집계다. 전체 64개 환경 중 낙상은 23개였다. 추종비의 유효 표본 수와 낙상 환경 수는 같은 분모를 뜻하지 않으므로 두 지표를 함께 해석해야 한다.

slow010 낙상은 foothold-v1 이 **0.359** 로 가장 나쁘다 (64 판 중 23 판). 전진 전용 레시피가 아주 느린 명령에서 무너지는 자리다. **추종비가 좋아 보이는 것과 같은 판에서 나온 숫자다.**

배포본 v2g2 를 같은 조건으로 켜다 (2026-09-28). v2b 와 한 글자도 다르지 않다 · env 64 · 시드 42 · pushes_enabled False · 지형 plane (무한 평면). 그래서 같은 표에 올릴 수 있다.

낙상 (64 env · 낮을수록 좋다)

판	slow010	turn_rest	turn_rev
NVIDIA	0.0000	0.0156	0.0781
foothold-v1	0.3594	1.0000	1.0000
D	0.0000	0.0312	0.0156
v2g2 @3000 (배포본)	0.0312	0.0781	0.1562

NVIDIA 와 foothold-v1 의 회전 두 칸을 2026-09-29 에 처음 켜다. 앞 판은 「안 켜다」로 비워 두고 있었다. 영상을 찍으면서 같은 조건으로 같이 돌렸다 (env 64 · 시드 42 · pushes_enabled False · 지형 plane).

foothold-v1 이 두 칸 모두 64/64 전판 낙상이다. 전진 전용 레시피가 제자리 회전에서 통째로 무너진다. 그래서 v2g2 의 turn_rev 0.1562 은 foothold-v1 대비로는 **1.0000 -> 0.1562** 이다.

그래도 D 보다는 나쁘다 (0.0156 · 한 판). 새로 켜 것이 닫는 물음은 「v1 과 NVIDIA 는 어떠했나」뿐이고, 「v2g2 의 회전이 충분한가」는 안 닫힌다.

turn_rev 0.1562 은 64 판 중 열 판이다. D 의 0.0156(한 판)보다 열 배 나쁘다. 반대로 도는 명령에서 무너지는 자리가 남아 있다. **판정 아홉 칸에는 turn_rev 가 없다.** 그래서 이 하락은 성적표에 안 나온다.

9-2-1. 확장 축 컷 열여덟 편

확장 평가의 저속 명령 네 가지와 회전 두 가지를 세 모델의 영상 18편으로 비교한다.

세 모델의 영상은 모두 env 51의 사례다. 캡션에서 해당 환경의 값과 전체 평가의 집계값을 구분한다. 사례 영상 하나가 64개 환경 전체를 대표하는 것은 아니다.

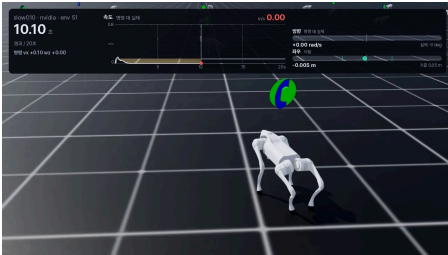
세 칸의 속도 축을 같게 맞췄다. 울타리마다 셋 중 가장 큰 축을 골라 같이 준다 (sim/eval/overlay/hud.py 의 y_top). 안 그러면 slow010 이 0.50 · 0.81 · 0.58 로 갈라져서 「명령선 아래로 얼마나 내려갔나」를 칸끼리 못 건준다 확인됨.

저속 넷은 앞 5 초가 측정에서 빠진다 (skip_s 5.0). 가속 구간이다. 화면에는 20 초가 다 나오므로 5 초 뒤부터 보는 것이 표와 같은 구간이다.

저속 0.10 m/s · 두 판이 «반대 방향으로» 틀린다

볼 것: 연한 가로선(명령 0.10)과 흰 실선(실제)의 거리. 그 사이 베이지색 면적이 미달분이다. 세 칸의 속도 축은 0.81로 같다.

NVIDIA

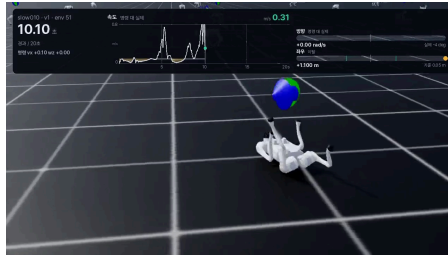


이 칸 실제 0.001 m/s · 추종비 0.01 · 20 초 내내 안 움직인다 (오른쪽 위 숫자가 0.00 에 붙어 있다) · 64 칸 추종비 0.06 · 낙상 0/64

V17 · 영상 보기 · 대표 장면 10.1초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

foothold-v1



이 칸 실제 0.053 m/s · 추종비 0.53 · 10.14 초에 넘어진다 (빨간 「넘어짐」이 뜨고 그래프가 멈춘다) · 64 칸 추종비 0.92 (추종비를 낼 수 있던 칸 48/64) · 낙상 23/64

V18 · 영상 보기 · 대표 장면 10.1초 / 20초

env 51에서 넘어지는 시점 부근의 실제 장면. 이후 초기화된 자세가 낙상 사례를 가리지 않도록 선정했다.

foothold-v2



이 칸 실제 0.174 m/s · 추종비 1.74 · 명령선 위로 계속 넘어선다 · 64 칸 추종비 1.67 · 낙상 2/64

V19 · 영상 보기 · 대표 장면 10.1초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

저속 0.20 m/s

NVIDIA



이 칸 실제 0.028 m/s · 추종비 0.14 · 64 칸 추종비 0.30 · 낙상 0/64

V20 · 영상 보기 · 대표 장면 12초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

foothold-v1

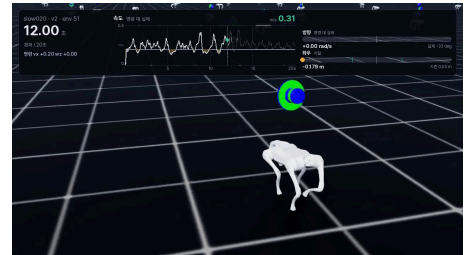


이 칸 실제 0.200 m/s · 추종비 1.00 · 이 울타리에서 명령선에 가장 가깝다 · 64 칸 추종비 1.00 · 낙상 0/64

V21 · 영상 보기 · 대표 장면 12초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

foothold-v2



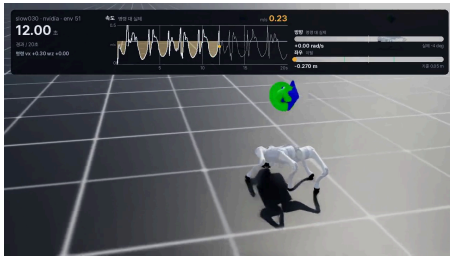
이 칸 실제 0.236 m/s · 추종비 1.18 · 64 칸 추종비 1.33 · 낙상 0/64

V22 · 영상 보기 · 대표 장면 12초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

저속 0.30 m/s

NVIDIA

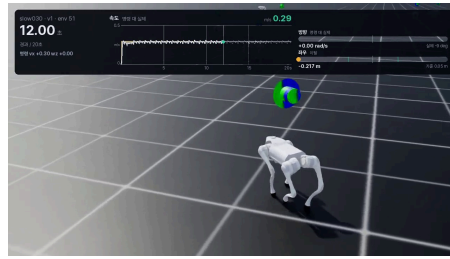


이 칸 실제 0.199 m/s · 추종비 0.66 · 64 칸 추종비 0.71 · 낙상 0/64

V23 · 영상 보기 · 대표 장면 12초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

foothold-v1

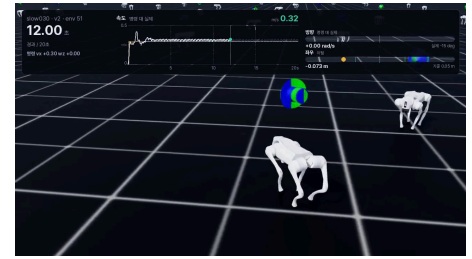


이 칸 실제 0.281 m/s · 추종비 0.94 · 64 칸 추종비 0.94 · 낙상 0/64

V24 · 영상 보기 · 대표 장면 12초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

foothold-v2



이 칸 실제 0.309 m/s · 추종비 **1.03** · 64 칸 추종비 1.04 · 낙상 0/64

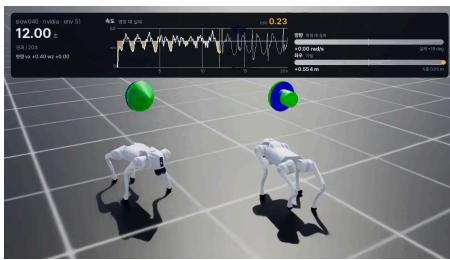
V25 · 영상 보기 · 대표 장면 12초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

저속 0.40 m/s · 학습 명령 하한과 같은 속도

0.40 m/s는 배포본 학습 명령의 전진 속도 하한이다. 이 평가에서 세 모델의 평균 추종비는 0.87~0.94였다. 더 낮은 명령보다 모델 간 차이가 작았지만, 학습 구간 진입만으로 차이가 사라진다고 단정할 수는 없다.

NVIDIA

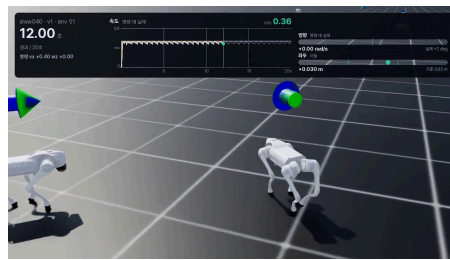


이 칸 실제 0.352 m/s · 추종비 0.88 · 64 칸 추종비 0.87 · 낙상 0/64

V26 · 영상 보기 · 대표 장면 12초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

foothold-v1

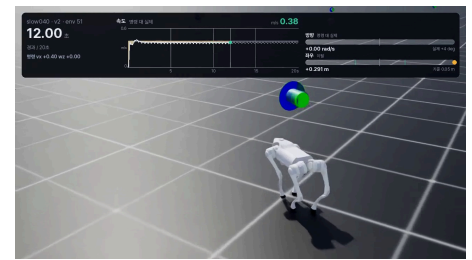


이 칸 실제 0.371 m/s · 추종비 0.93 · 64 칸 추종비 0.93 · 낙상 0/64

V27 · 영상 보기 · 대표 장면 12초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

foothold-v2



이 칸 실제 0.372 m/s · 추종비 0.93 · 64 칸 추종비 0.94 · 낙상 0/64

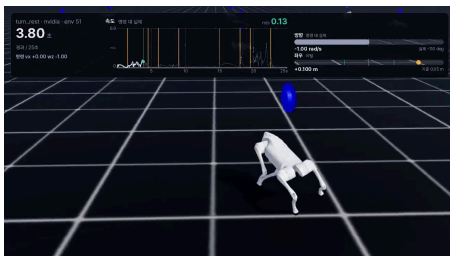
V28 · 영상 보기 · 대표 장면 12초 / 20초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

turn_rest · 제자리 회전 계단 · 칸 사이 1.5 초 쉼

볼 것: 주황 세로선이 명령 계단 경계다. 선 두 개가 붙어 있는 자리가 쉼 구간이다. 오른쪽 「방향」칸의 실제 N deg 가 누적 회전이다.

NVIDIA

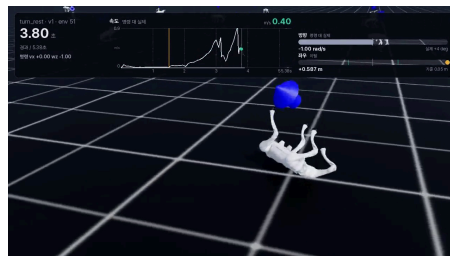


이 칸 요레이트 추종비 -1.00 에서 0.84 · +1.00 에서 0.62 · 64 칸 낙상 **1/64**

V29 · 영상 보기 · 대표 장면 3.8초 / 25초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

foothold-v1

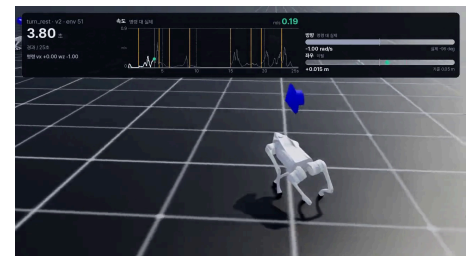


3.92 초에 넘어진다. 첫 계단(-1.00)에서 추종비가 -0.26 으로 반대로 돈다 · 64 칸 낙상 **64/64 (전판)**

V30 · 영상 보기 · 대표 장면 3.8초 / 5.38초

env 51에서 넘어지는 시점 부근의 실제 장면. 이후 초기화된 자세가 낙상 사례를 가리지 않도록 선정했다.

foothold-v2



이 칸 요레이트 추종비 -1.00 에서 0.73 · +1.00 에서 0.44 · 64 칸 낙상 **5/64**

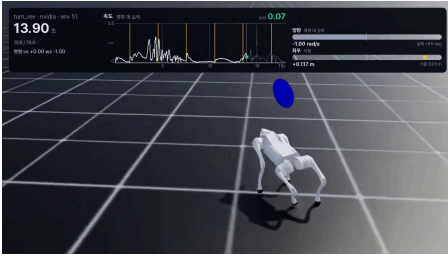
V31 · 영상 보기 · 대표 장면 3.8초 / 25초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

turn_rev · 같은 계단을 순서만 뒤집음 (+1.0 이 처음)

이 칸이 v2g2 가 가장 나쁜 자리다 (10/64). 그래도 foothold-v1 은 64/64 다.

NVIDIA

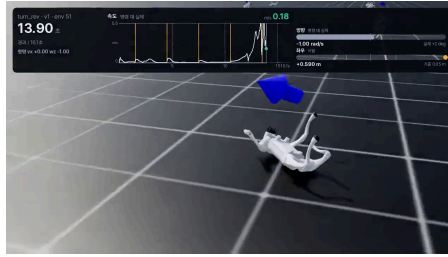


이 칸 요레이트 추종비 -1.00 에서 0.70 · +0.50 에서 0.23 · 64 칸 낙상 **5/64**

V32 · 영상 보기 · 대표 장면 13.9초 / 18초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

foothold-v1

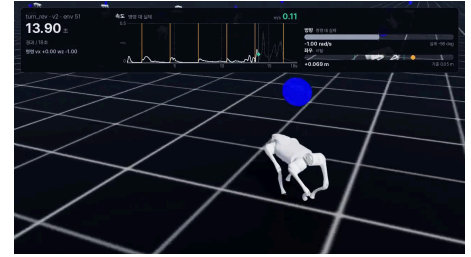


13.96 초에 넘어진다. +0.50 에서 추종비 -0.00 · +1.00 에서 0.01 로 **아예 안 돈다** · 64 칸 낙상 **64/64 (전판)**

V33 · 영상 보기 · 대표 장면 13.9초 / 16.1초

env 51에서 넘어지는 시점 부근의 실제 장면. 이후 초기화된 자세가 낙상 사례를 가리지 않도록 선정했다.

foothold-v2



이 칸 요레이트 추종비 -1.00 에서 0.67 · +0.50 에서 0.00 · **양의 요레이트를 못 따른다** · 64 칸 낙상 **10/64**

V34 · 영상 보기 · 대표 장면 13.9초 / 18초

같은 env 51의 명령 응답 장면. 화면의 순간값과 캡션의 평가 구간 평균은 다를 수 있다.

turn_rev에서 +0.50 rad/s 명령의 평균 추종비는 NVIDIA 0.43, foothold-v1 약 0.00, v2g2 0.01이었다. 이 평가에서 양의 요레이트 추종이 약한 경향은 세 모델에 공통으로 나타났다. 다만 공통 경향만으로 학습 레시피의 영향을 배제할 수는 없다. 원인은 통제 실험으로 추가 확인해야 한다.

외란 조건의 비교 평가 결과는 아직 제시하지 않는다. 외란 이벤트를 유지하는 옵션과, 외란의 크기·시점·판정 기준을 통제하는 독립 평가 시나리오는 구분해야 한다.

9-3. 앞으로 축 2 를 어떻게 넓힐 것인가

현재 판정 아홉 항목은 평지에서 명령 수행을 평가한다. 횡이동·힘지 회전·장거리는 평가 하네스의 확장이 필요하다. 외란 이벤트 유지 옵션은 있지만, push_robot이라는 독립 평가 시나리오는 등록돼 있지 않다. 아래 표에서 구현과 측정의 부족을 구분한다.

넓힐 축	무엇을 묻나	왜 아직 없나 (근거)	만들려면
횡이동 (vy)	옆으로 가라는 명령을 따르나	시나리오 열 개가 모두 vy 를 0.0 으로 고정 한다 확인됨 (sim/eval/eval_command_response.py 157 ~ 262 행의 프로필 열 개를 0.1 초 간격으로 돌려 보니 가운데 값이 전부 0.0)	프로필에 vy 를 열고, 문턱을 결과를 보기 전에 정한다
힘지 회전	평지가 아니라 힘지에서 제자리 회전이 되나	축 2 프로브는 평면 지형을 사용한다. sim/eval/eval_command_response.py의 환경 구성에서 terrain_type을 plane으로 지정한다.	축 2 하네스가 --terrain 을 받게 하고 축 1 의 지형 등록(sim/eval/terrains.py) 을 재사용한다
장거리	20 초가 아니라 분 단위로 가도 안 무너지나	가장 긴 시나리오가 25 초 다 (turn_rest)	episode_length_s 를 늘리고 드리프트 누적을 켜다 (지금은 끝 1 초만 본다)
push_robot	외란을 견디나	--keep_pushes는 기존 외란 이벤트를 유지하는 옵션이다. SCENARIOS에는 push_robot이 등록돼 있지 않으며, 비교 평가 결과도 없다.	외란의 크기·방향·시점과 회복 지표를 명세하고, 이벤트 발생을 검증한 뒤 NVIDIA.foothold-v1·배포본을 같은 조건으로 평가한다.

문턱을 결과 뒤에 정하지 않는다. 이것이 넷을 판정에 안 넣고 기록만 하는 까닭이고, 9-2 의 저속 표에서 **자를 잘못 골라 결론이 뒤집혔던 것** 과 같은 자리다. 자를 먼저 정하고 그 자가 무엇을 재는지 적어야 한다.

외란 평가는 이벤트의 크기·방향·시점과 회복 지표를 먼저 정하고, 실제 외란이 발생하는지 확인한 뒤 기준선 세 모델을 같은 조건으로 비교한다. 횡이동은 명령 프로필과 판정 기준을, 힘지 회전과 장거리는 하네스의 지형·시간 설정을 확장해야 한다.

10. 배포 모델과 참고 비교군

배포는 v2g2 iter3000. 강력한 참고 비교군은 v2b-r iter2500.

	v2b-r @2500	v2g2 @3000	시드 폭
48 칸 평균 (d0.5)	93.12	94.19	1.25 %p
미경험 10종	89.10	90.70	
셋 다 학습 6종	99.83	100.00	
축 1 진짜 하락 (그 점)	0	0	
축 1 진짜 상승 (그 점)	5	8	
축 1 네 점 하락	2 · 0 · 0 · 0	0 · 0 · 0 · 0	
축 2 (1024 env)	9 / 9	8 / 9	4 칸
stepping_stones 1.0 m/s	0 %	24 %	
시드 재현	없음	있음 (43 · 44)	

두 판은 성적으로 구별되지 않는다. 48 칸 차이 1.07 %p 는 시드 폭 안이고, 축 2 의 한 칸 차이도 시드 폭 네 칸 안이다.

v2g2 를 고른 근거는 「더 잘한다」가 아니다. 축 1 네 점에서 v2b-r 은 iter1500 에 하락 2 가 있고 v2g2 는 넷 다 0 이다. 체크포인트를 골라야 하는 부담이 v2g2 쪽이 작다. 그리고 v2g2 만 다른 시드로 재현을 재 봤다.

v2b-r 의 9/9 는 네 점 중 한 점이고 회전 낙상이 0.344 -> 0.109 -> **0.031** -> 0.3125 로 한 점만 내려앉는다. 봉우리다.

11. 관측과 정책의 다음 연구 과제

여기까지 무엇을 세웠고, 다음은 어디를 건드리나

정책 수준(보상·명령)의 탐색은 멈출 선에 닿았다. 다음은 관측이다

① 지금 정책이 보는 것

env.yaml 의 observations.policy 여덟 항목

base_lin_vel	joint_pos
base_ang_vel	joint_vel
projected_gravity	actions
velocity_commands	height_scan

여기가 다음 자리

② 신경망을 바꿔도 남는 것

평가 하네스	지형에 올리고 행동만 켜다. 관측이 무엇이든 상관없다
기준선 숫자	새 구조가 넘어야 할 선. 없으면 «더 낮다» 를 반증 못 한다
환경의 함정	커리큘럼 붕괴 · <code>ang_vel_z</code> 와 <code>heading</code> 의 한 쌍
명령·보상 설정	새 구조도 명령을 받고 보상으로 배운다
멈출 기준	시드 폭보다 작은 차이는 차이가 아니다

③ 다음 단계가 답할 물음

관측을 무엇으로 채울까	지금은 발밑 격자를 직접 읽는다
어떻게 압축할까	격자 그대로 넣을까 · 요약해 넣을까
기억을 둘까	지금은 한 시점만 본다
못 본 자리를 어떻게 메울까	가려진 곳의 값을 무엇으로 둘까

버려지는 것은 가중치 파일 하나다. 관측 차원이 바뀌면 `resume` 이 안 되므로 체크포인트는 못 쓴다.
그 외는 남는다. 그래서 신경망 교체가 예정돼 있으면 정책 탐색은 멈출 선에 닿는 순간 멈추는 것이 맞다.

그림 7 · 지금 정책이 보는 여덟 항목, 신경망을 바꿔도 남는 것, 다음 단계가 답할 물음

지금 정책이 보는 것



그림 8 · 실제 촬영 위에 그린 height_scan. 몸통 밑 1.6 x 1.0 m 를 0.1 m 간격 187 점으로 훑습니다. 레일 모서리가 스캔 범위를 가로지릅니다

그림은 실제 촬영 위에 실적으로 그린 것이다. 카메라를 trace 에서 그대로 세워 점마다 투영했고 (target 이 화면 정중앙으로 떨어지는 것으로 검산했다), 격자 규격은 params/env.yaml 의 height_scanner 에서 읽었다.

187 이라는 수를 체크포인트로 반증했다. actor.0.weight 가 (512, 235) 이고 몸통·관절이 3+3+3+3+12+12+12 = 48 이므로 235 - 48 = 187 이다 확인됨.

params/env.yaml 의 observations.policy 여덟 항목이다.

항목	무엇인가
base_lin_vel	몸통이 지금 나아가는 속도 (앞뒤 · 좌우 · 위아래)
base_ang_vel	몸통이 지금 도는 속도 (구르기 · 끄덕임 · 요)
projected_gravity	중력이 몸통 기준으로 어느 쪽인가. 기울기를 아는 자리
velocity_commands	지금 받은 명령 (앞으로 얼마 · 옆으로 얼마 · 얼마나 돌려)
joint_pos	관절 열둘이 지금 어디에 있나 (기준 자세에서의 차이)
joint_vel	관절 열둘이 지금 얼마나 빨리 움직이나
actions	바로 앞 걸음에 내보낸 관절 목표각
height_scan	발밑 격자에 광선을 쏘아 켜 바닥 높이. 지형 정보는 이것 하나뿐이다

앞 일곱은 몸통과 관절의 상태다. 지형을 아는 통로는 height_scan 하나다.

신경망을 바꾸면 이 일이 버려지나

버려지는 것은 가중치 파일 하나다. 관측 차원과 뜻이 바뀌면 resume 이 안 되므로 체크포인트는 못 쓴다. 그 외는 남는다.

남는 것	왜
평가 하네스	정책을 지형에 올리고 행동만 잔다. 관측이 무엇이든 상관없다
기준선 숫자	새 구조가 넘어야 할 선. 없으면 「더 낫다」를 반증할 수 없다
환경의 함정	커리큘럼 붕괴 · <code>ang_vel_z</code> 와 <code>heading</code> 의 한 쌍 · 노출 구조
명령·보상 설정	새 구조도 명령을 받고 보상으로 배운다. 출발 설정이 된다
멈출 기준	시드 폭보다 작은 차이는 차이가 아니다

진짜로 버려지는 것은 「정책 수준에서 마지막 몇 %p 를 더 짜내는 시간」이다. 그래서 신경망 교체가 예정돼 있으면 규칙 1 에 닿는 순간 멈추는 것이 맞다.

다음에 주목할 것 여섯

무엇	왜
가 관측을 어떻게 다룰 것인가	지형 토폴로가 <code>height_scan</code> 하나다. 격자 그대로 넣을까 · 요약해 넣을까 · 기억을 둘까 · 가려진 곳의 값을 무엇으로 둘까
나 축 2 확장 축을 v2 부터 쌓는다	저속 명령과 <code>turn_rest</code> · <code>turn_rev</code> 는 배포본까지 평가했다. 횡이동·험지 회전·장거리 하네스는 확장이 필요하다. 외란은 이벤트 유지 옵션만으로 독립 평가가 완성되지 않으므로 조건과 회복 지표를 먼저 정의한다. <code>turn_rev</code> 의 낙상 0.1562를 포함한 확장 평가 결과와 사례 영상은 9절에서 제시한다.
다 축 2 의 자를 고친다	값과 Wilson 구간을 같이 적는다 (비용 0). 표본을 1024 로 올린다
라 resume 실험을 재개한다	중단된 실험에 공통으로 있던 resume의 영향을 분리해야 한다. 초기 가중치, optimizer 상태, 명령·지형 조건을 통제해 비교한다. 아직 원인이 확인된 것은 아니다.
마 난이도 구멍을 메운다	0.8 이 아무 판에도 없었다. 이번에 v1 · v2 열두 칸을 새로 썼다
바 지형별 성공률이 «지형 판 하나» 의 값인지 확인한다	평가 설정이 <code>num_rows=1</code> · <code>num_cols=10</code> 이라 지형마다 8 x 8 m 한 칸이고 env 열 개가 그 한 판을 나눠 쓴다. 곧 100 판이 한 레이아웃에 스폰 흔들림 100 번이다. <code>stepping_stones</code> 1.0 m/s 에서 그 칸의 전진 성공률이 45 % 인데, 지형을 새로 만들어 찍은 열다섯 판이 0/15 였다 (확률 0.0001 · 13 절). 다른 설정은 전부 같은 것을 확인했고 「모서리 칸이 어렵다」가설은 기각했다. <code>num_rows</code> 를 올려 여러 판에서 재 보면 갈린다. 갈리면 축 1 의 지형별 숫자에 판 편차를 같이 적어야 한다

`stepping_stones` 가 「가」의 시험지다. 정책 수준에서 여덟 종 중 일곱을 풀었는데 이것 하나가 8.0 % 로 남았다. 발 디딜 곳이 띄엄띄엄한 지형이라 「발밑 한 점의 높이」로는 부족한 것이 아닌가를 검증할 자리다. 아직 그 가설을 시험하지 않았다 미측정.

12. 재현 방법

평가 (축 1 · 한 칸)

```
python sim/eval/eval_generalization.py \
  --checkpoint <정책.pt> --terrain_set unseen10 --terrains all \
  --difficulty 0.5 --episodes 100 --envs_per_terrain 10 \
  --command_vx 1.0 --eval_duration 6.0 \
  --min_progress_m 3.0 --max_lateral_drift 0.75 \
  --seed 42 --headless --output_dir <결과폴더>
```

판정 (축 2 · 네 점)

```
python sim/eval/axis2_fourpoint.py --root <축2결과> --policies v2g2-feetair01
```

표 다시 굽기

```
python _out/loop/v2_sweep.py --out sim/eval/results/20260928-v2-sweep
```

도식 다시 굽기

```
python tools/make_v2_figs.py --write
python tools/svg_selfcontained.py --dir docs/assets/visual --write
python tools/svg_theme.py --write
```

도식의 숫자는 손으로 적은 것이 하나도 없다. 녀 장 전부 sweep_long.csv 를 읽어서 그린다. 표가 바뀌면 그림도 같이 바뀐다.

13. 영상으로 확인하는 결과와 한계

각 영상은 지형 통과, 명령 응답, 남은 실패를 확인하기 위한 사례다. 개별 장면의 결과와 전체 평가에서 집계한 비율을 구분해 제시한다.

축 1 · 48 칸 (갤러리)

지형 열여섯 중 × 속도 셋. foothold-v1 판의 기준선 둘(NVIDIA · v1)과 같은 자리를 같은 카메라로 찍었으므로 세 열을 나란히 견줄 수 있다.

어디서 보나	무엇
갤러리 · v2/gallery/	48 컷 전부 · 평가 칸 144
판 비교 (3열)/gallery/compare/	NVIDIA · v1 · v2 를 같은 칸에서

기본으로 뜨는 칸은 rails 1.0 이다. 세 판이 8 · 48 · 100 으로 가장 크게 갈리는 자리다 (48 칸 전부를 「세 값의 이웃 간 최소 간격」으로 줄 세워 골랐다). 다만 rails 는 v2 가 학습에 넣은 지형이므로 그 칸은 일반화가 아니라 학습을 보여 준다. 화면에도 그렇게 적는다.

축 2 · 명령 응답 · 세 판을 같은 컷으로

여기가 v2 의 핵심이다. 축 1(전진 성공률)만 보면 foothold-v1 은 좋은 판이다. 그런데 명령을 주면 무너진다. 같은 시나리오를 세 판에 걸어 봤다.

시나리오 · 값	NVIDIA	foothold-v1	foothold-v2
stop 낙상	0.0000	0.2031	0.0000
hold 낙상	0.0000	0.0781	0.0000
hold 잔류 속도 (m/s)	0.0011	0.0240	0.0005
hold 관절 떨림	0.0015	0.1258	0.0003

시나리오 · 값	NVIDIA	foothold-v1	foothold-v2
turn 낙상	0.0625	1.0000	0.1094

foothold-v1 의 turn 은 **64 판이 64 판 다 넘어진다**. 전진 성공률만 보고 달렸을 때 무엇을 잃었는지가 이 한 칸에 있다.

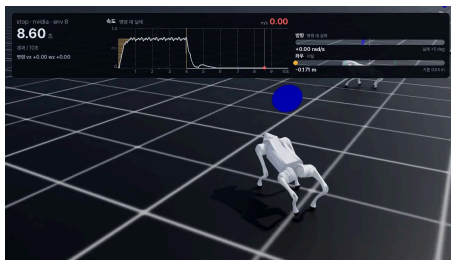
세 모델의 영상은 모두 env 8의 사례다. foothold-v1이 stop-hold-turn에서 넘어지는 사례를 골라 같은 환경 번호로 비교했다. 이 장면은 대표 사례이며, 전체 64개 환경의 낙상 비율은 별도 표로 제시한다.

env 8	NVIDIA	foothold-v1	foothold-v2
stop	안 넘어짐	넘어짐	안 넘어짐
hold	안 넘어짐	넘어짐	안 넘어짐
turn	안 넘어짐	넘어짐	안 넘어짐

stop · 4 초 전진 뒤 명령 0

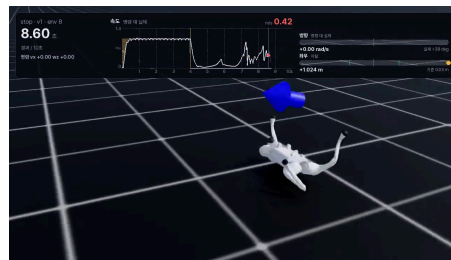
볼 것: 명령이 0 이 된 뒤에도 미끄러져 나가는가.

NVIDIA



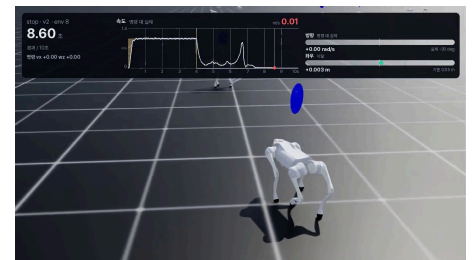
env 8 안 넘어짐 · 64 판 낙상 0.0000
V35 · 영상 보기 · 대표 장면 8.6초 / 10초
정지 명령 이후 자세. v1 낙상 장면과 같은 시점

foothold-v1



env 8 넘어짐 · 64 판 낙상 0.2031 (13 판)
V36 · 영상 보기 · 대표 장면 8.6초 / 10초
정지 명령 이후 낙상. 리셋 이전

foothold-v2



env 8 안 넘어짐 · 64 판 낙상 0.0000
V37 · 영상 보기 · 대표 장면 8.6초 / 10초
정지 명령 이후 자세. v1 낙상 장면과 같은 시점

hold · 20 초 내내 명령 0

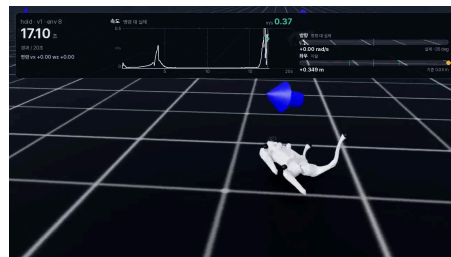
볼 것: 선 채로 떨어지 않는가. 관절 떨림이 foothold-v1 0.125841 대 foothold-v2 0.000257 로 **490 배**다 (위 표는 넷째 자리에서 끊은 값이라 그것끼리 나누면 419 가 나온다. 배율은 원자료로 켤 것이다). 화면에서는 발을 계속 고쳐 딛는 것으로 보인다.

NVIDIA



env 8 안 넘어짐 · 64 판 낙상 0.0000 · 관절 떨림 0.0015
V38 · 영상 보기 · 대표 장면 17.1초 / 20초
정지 유지 후반부. v1 낙상 장면과 같은 시점

foothold-v1



env 8 넘어짐 · 64 판 낙상 0.0781 (5 판) · 관절 떨림 0.1258
V39 · 영상 보기 · 대표 장면 17.1초 / 20초
정지 유지 중 옆으로 넘어진 자세. 리셋 이전

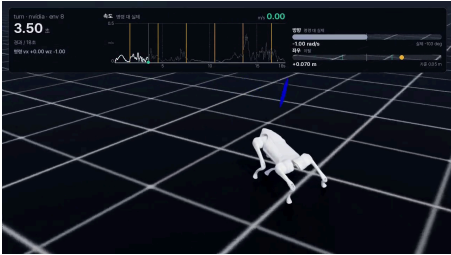
foothold-v2



env 8 안 넘어짐 · 64 판 낙상 0.0000 · 관절 떨림 0.0003
V40 · 영상 보기 · 대표 장면 17.1초 / 20초
정지 유지 후반부. v1 낙상 장면과 같은 시점

turn · 제자리 요레이트 계단 다섯
볼 것: 도는가, 아니면 자빠지는가.

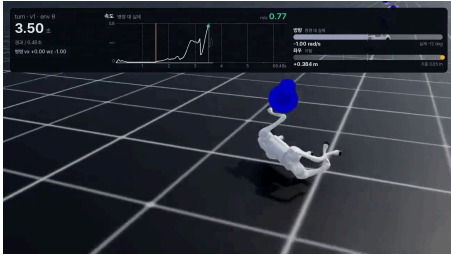
NVIDIA



env 8 안 넘어짐 · 64 판 낙상 0.0625 · 계단 다섯을 다 돈다

V41 · 영상 보기 · 대표 장면 3.5초 / 18초
첫 회전 구간. v1 낙상 장면과 같은 시점

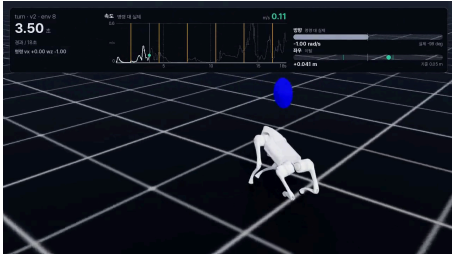
foothold-v1



env 8 넘어짐 · 64 판 낙상 1.0000 (전부) · 대표 장면 3.5초에서 넘어지는 모습이 보인다. 낙상 판정은 3.70초이며, 전체 영상 길이는 6.48초다. 이후 환경이 초기화된다

V42 · 영상 보기 · 대표 장면 3.5초 / 6.48초
첫 회전 구간에서 넘어지는 장면. 리셋 이전

foothold-v2



env 8 안 넘어짐 · 64 판 낙상 0.1094 · 계단 다섯을 다 돈다

V43 · 영상 보기 · 대표 장면 3.5초 / 18초
첫 회전 구간. v1 낙상 장면과 같은 시점

정지 전환 영상은 10초, 정지 유지 영상은 20초다. 회전 영상은 NVIDIA와 foothold-v2가 18초, foothold-v1이 6.48초다. 영상 길이와 환경의 낙상 판정 시각은 구분한다.

아직 못 넘는 것 · 되레 나빠진 것

볼 것: 무엇을 못 하는가. stepping_stones 는 미경험 여덟 중 유일하게 못 넘는다 (8.0 %). 디딜 곳이 점점이 떨어져 있어 발을 «어디에» 놓을지가 문제인데, 지금 정책이 지형을 아는 통로는 height_scan 하나다 (11 절).

8.0 % 는 세 속도 평균이다. 속도마다 성분까지 퍼면 이렇다 확인됨.

속도	종합	생존	전진	추중	방향
0.5 m/s	0 %	36 %	7 %	49 %	8 %
1.0 m/s	24 %	48 %	45 %	31 %	45 %
1.5 m/s	0 %	53 %	23 %	0 %	24 %

매끄러운 속도 추세가 아니다. 느려도 0 이고 빨라도 0 인데 가운데만 24 % 다. 「천천히 가면 디딜 곳을 고를 여유가 생긴다」가 아니라는 뜻 이다. 0.5 m/s 는 전진이 7 % 로 아예 못 나아가고, 1.5 m/s 는 전진 23 % 인데 추중이 0 % 라 종합이 0 이 된다. 곧 같은 0 % 가 서로 다른 까닭이다.

통과선은 세 속도 다 3.0 m 다 (이상 거리 6 m × min_progress_ratio 0.5). 시간이 속도에 맞춰 12 · 6 · 4 초라 이상 거리가 언제나 6 m 다.

난이도 0.5 와 0.6 사이에 벼랑이 있다 확인됨 (1.0 m/s 칸).

난이도	종합	생존	전진
0.1	100 %	100 %	100 %
0.2	24 %	44 %	38 %
0.3	20 %	51 %	41 %
0.4	24 %	49 %	41 %
0.5	24 %	48 %	45 %
0.6	0 %	52 %	0 %
0.7 ~ 0.9	0 %	32 ~ 47 %	0 %

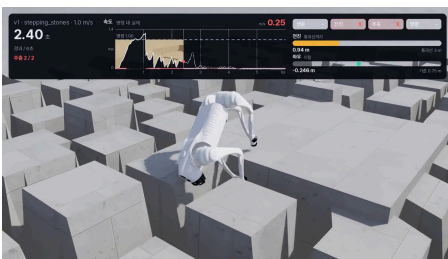
0.1에서는 세 속도 다 100 % 다. 그리고 0.6 부터는 전진이 0 % 로 한 걸음도 못 나아간다 (생존은 여전히 절반이라 «넘어져서» 가 아니다).

곧 「stepping_stones 를 못 넘는다」는 한 문장이 세 가지 이야기다.

난이도	무엇이 막는가
0.1	막는 것이 없다
0.2 ~ 0.5	속도다. 1.0 m/s 에서만 건너고 0.5 와 1.5 는 0 % 다
0.6 이상	지형이다. 속도를 바꿔도 전진이 0 % 다

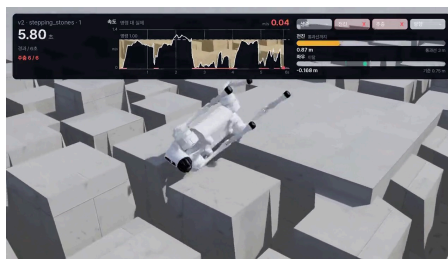
축 1 은 난이도 0.5 에서만 재므로 이 버량도 성적표에 안 나온다. pyramid_stairs_inv 의 87 %p 하락과 같은 부류다.

foothold-v1 · 1.0 m/s



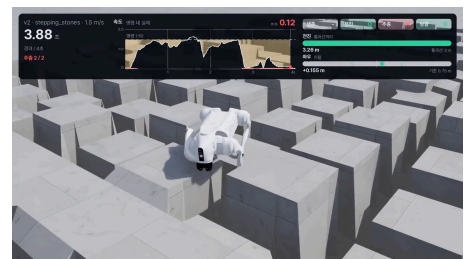
못 건넌다 · 전진 0.93 m · 그 칸 전진 성공률 0 %
V44 · 영상 보기 · 대표 장면 2.4초 / 6초
희소 발판 사이에서 진행하지 못하는 장면

foothold-v2 · 1.0 m/s



못 건넌다 · 전진 0.86 m · 그 칸 전진 성공률 45 %
· 종합 24 %
V45 · 영상 보기 · 대표 장면 5.8초 / 6초
희소 발판에서 몸이 기울고 통과하지 못하는 장면

foothold-v2 · 1.5 m/s



건넌다 · 전진 3.26 m · 다만 종합은 0 % 다 (추종 0 %)
V46 · 영상 보기 · 대표 장면 3.88초 / 4초
통과선 이후의 위치. 통과와 속도 추종 성공은 구분

셋째 칸을 「1.5 m/s」로 둔 까닭. 영상의 목적은 v2의 통과 사례를 보여 주는 것이었다. 1.0 m/s 에서 건너는 판을 찍으려고 시드 열다섯을 훑었으나 0 / 15 였다 (전진 0.01 ~ 2.35 m). 그 칸의 전진 성공률이 45 % 이므로 0/15 는 확률 0.0001 이다. 무언가 체계적으로 다르다.

같은 것을 하나씩 확인했다. 통과선 3.0 m · 시간 6.0 초 · 스폰 ± 0.10 m · 요 ± 5 도 · gap_aware_scan True · height_scan_miss_value 1.0 · difficulty_range [0.5, 0.5] · dt 0.02 · eval_spec_version 2 · 체크포인트 파일. 전부 같다.

가설 하나를 세워 재 보고 **기각했다**. 「8x8 격자의 모서리 칸(-28, -28)이 유난히 어렵다」. 격자를 1x1 로 두어 자리를 (0, 0) 으로 옮기고 같은 시드 셋을 돌리니 1.10 · 0.93 · 2.35 m 었다 (모서리에서는 0.86 · 0.89 · 2.09 m). **자리 탓이 아니다**.

남은 차이는 **지형 «판»** 이다. 평가 설정은 num_rows=1 · num_cols=10 이라 stepping_stones 가 8 x 8 m **한 칸**이고, env 열 개가 그 한 판을 나눠 쓴다. 곧 **100 판은 「한 레이아웃에 스폰 흔들림 100 번」** 이다. 반면 녹화기는 지형 하나로 걸러 격자를 새로 만드므로 **시드마다 새 레이아웃**이 나온다.

그래서 두 숫자는 서로 다른 것을 재고 있다. 45 % 는 그 한 판에서의 값이고, 내 0/15 는 열다섯 판에서의 값이다. 이것이 참이면 **축 1** 의 지형별 성공률은 **«그 한 판의 운»** 을 안고 있다. 확인하지 못했으므로 단정하지 않고 **열어 둔 물음으로 남긴다** (11 절).

세 속도 전부는 갤러리/gallery/ 에서 stepping_stones 를 고르면 볼 수 있다.

볼 것: 축 1 이 안 재는 자리에서 v2 가 v1 보다 나빠진다. pyramid_stairs_inv 난이도 0.9 · 1.5 m/s 에서 foothold-v1 이 90 % 인데 foothold-v2 는 3 % 다. **축 1 은 난이도 0.5 에서만 재므로 이 87 %p 하락은 성적표에 한 줄도 안 나온다**.

앞 판에 적었던 「실패하는 판을 골라 다시 굽는 것이 맞다」를 철회한다. 다시 구울 것이 없다. **걸린 이 컷이 바로 그 판이다**.

그때는 「전진이 둘 다 통과선을 넘었으니 하락을 못 보여 준다」고 읽었다. 종합 3 % 를 **«못 건넌다»** 로 읽은 것이 틀렸다. 성분을 퍼 보면 이렇다.

판	종합	생존	전진	추종	방향
foothold-v1	90 %	100 %	100 %	90 %	100 %
foothold-v2	3 %	100 %	99 %	3 %	98 %

생존 100 % · 전진 99 % 다. 넘어지지도 않고 건넌다. 3 % 로 내려온 것은 추종 한 칸이고, 종합이 AND 라 그 한 칸이 전체를 끌어내렸다. 곧 87 %p 하락은 **「못 건넌다」가 아니라 「명령 속도를 못 따라간다」** 다.

걸린 두 컷의 trace 를 재니 그 칸의 값과 맞는다.

판	명령	평균 속도	전진	추종 MAE	문턱 0.25
v1	1.50	1.30	4.56 m	0.209	안 넘는다 → 통과
v2	1.50	1.25	4.46 m	0.263	넘는다 → 실패

v2 컷은 예외가 아니라 97 % 쪽이다. 화면에서 볼 것은 「계단 코에 걸려 넘어지나」가 아니라 **속도 그래프가 명령선 1.50 아래에 얼마나 머무나** 다.

HUD 가 그것을 세 곳에서 보여 준다 확인됨 (v2 컷 3.60 초 자리).

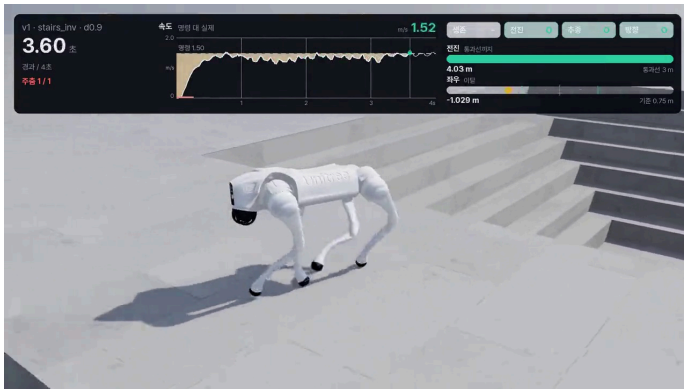
화면의 어디

무엇을 말하나

오른쪽 위 램프 넷	생존 - · 전진 초록 · 추종 빨간 X · 방향 초록
전진 막대	그 자리 3.91 m (끝값 4.46 m) · 통과선 3 m 를 넘어 꼭 찼다
속도 차트	명령선 1.50 아래 베이지색 면적이 미달분이다

곧 떨어진 칸이 화면에 빨간 X 하나로 찍혀 있다. 「성공한 영상 같은데」로 보이는 것이 맞고, 그것이 이 칸의 이야기다 (건너지만 못 따라간다).

foothold-v1 · d0.9 · 1.5 m/s

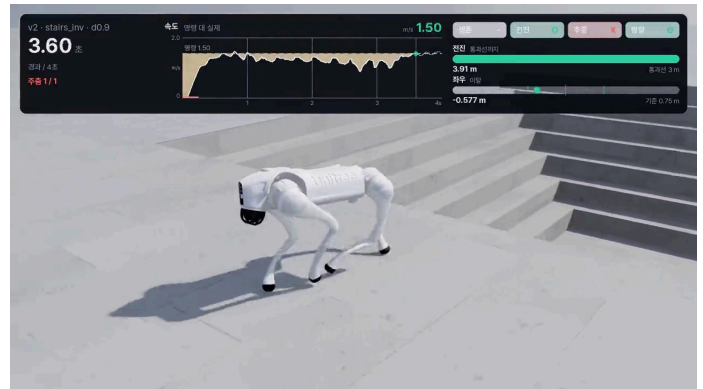


명령 1.50 m/s · 종합 90 % = 추종 90 % · 이 판은 전진 4.56 m 로 건넌것 추종 MAE 0.209 로 문턱 0.25 아래 · 램프 넷이 다 초록이다

V47 · 영상 보기 · 대표 장면 3.6초 / 4초

계단을 지난 뒤의 자세와 추종 HUD 캡션의 전진 거리는 영상 전체의 최종값이며, 대표 장면 시각의 HUD 값과 다를 수 있다.

foothold-v2 · d0.9 · 1.5 m/s



명령 1.50 m/s · 종합 3 % = 추종 3 % · 이 판도 전진 4.46 m 로 건넌것 안 넘어졌다 · 추종 램프만 빨간 X 이고 속도 차트의 명령선 1.50 아래 베이지색 면적이 그 미달이다 (MAE 0.263)

V48 · 영상 보기 · 대표 장면 3.6초 / 4초

넘어지지 않고 건넌것지만 속도 추종이 미달한 HUD 캡션의 전진 거리는 영상 전체의 최종값이며, 대표 장면 시각의 HUD 값과 다를 수 있다.

학습 진행 · 41.3 초 · 대군이 여러 험지에서

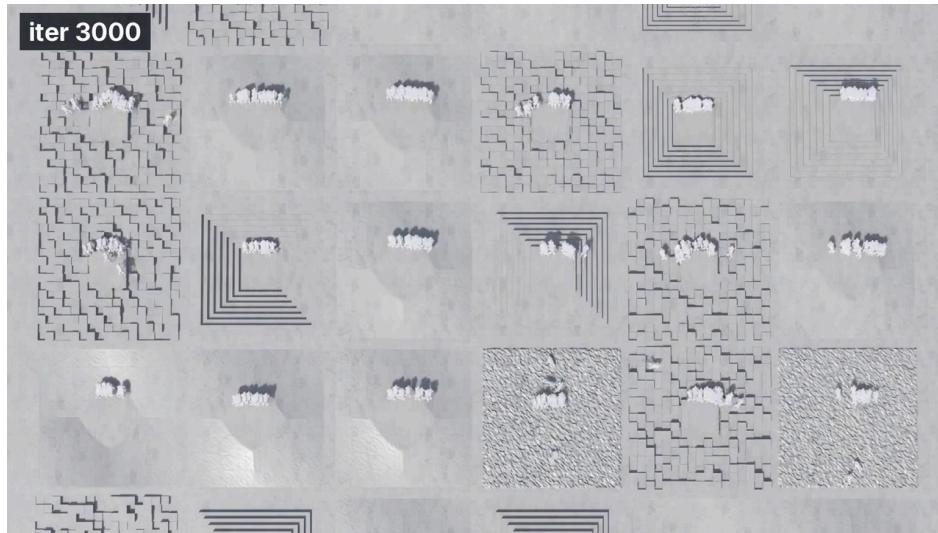
볼 것: 무리가 흩어져 해매던 것이 한 덩이로 뭉쳐 같이 나아가게 된다.

같은 격자 · 같은 카메라에서 정책만 같아 끼운다. 저장본 열여섯을 고르게 뽑았다 (0 · 200 ~ 2800 · 3000). 조각마다 왼쪽 위에 iter N 이 태워져 있다.

격자는 --terrain all --terrain_set rough6 --terrain_cols 6 --terrain_rows 5 이고 --max_init_level 4 다. 곧 열마다 다른 하위 지형이 오고 로봇이 다섯 행에 퍼진다. 한 화면에 여섯 지형 x 다섯 난이도 층의 무리가 같이 나온다.

학습에서는 4,096개 환경을 200개 지형 칸에 배치해 칸당 약 20개 환경을 사용한다. 이 영상은 30개 칸에 600개 환경을 배치해 비슷한 밀도로 재생했다. 환경 간 물리 상호작용은 격리돼 있다. 영상은 저장된 정책을 재생한 시각 자료이며 학습 전체를 실시간 촬영한 것은 아니다.

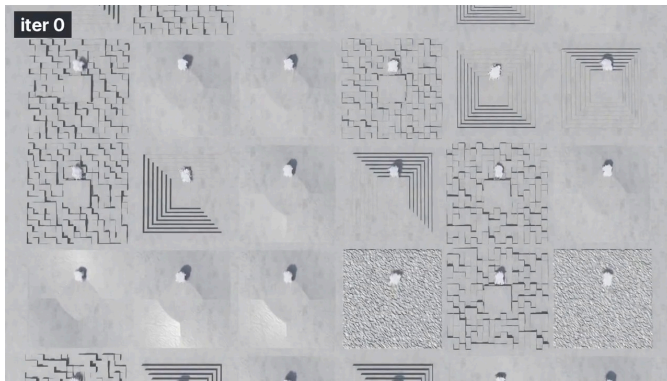
v2 학습 진행



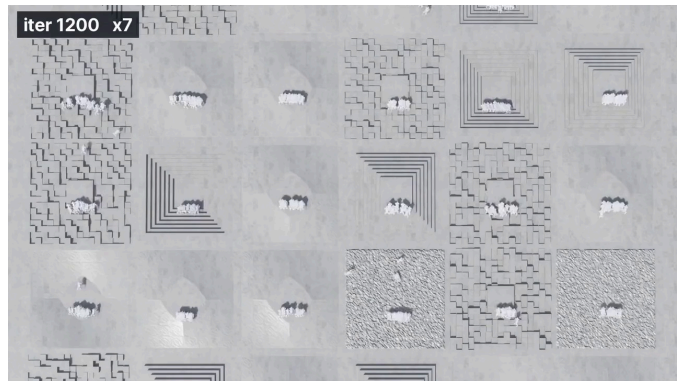
험지 여섯 중 30 칸 위 **600 마리** (칸당 약 20 마리로 학습과 같은 밀도) · 저장본 열여섯 · **iter 0 과 3000 은 정속** · 가운데는 5 배에서 10 배로 빨라진다 (왼쪽 위에 iter N x5 처럼 적어 둔다)

V49 · 영상 보기 · 대표 장면 39초 / 41.28초

저장 정책 iter 3000의 보행 장면. 추가 장면은 초기 정책과 학습 중간을 비교하며, 중간 구간은 표시된 배속으로 재생된다.



학습 진행 비교 · 2초



학습 진행 비교 · 20초

이 영상은 「못 걷다가 걸게 된다」가 아니다. 7 절에 적었듯 v2 는 NVIDIA 가중치에서 이어 학습했으므로 model_0 이 이미 걷는다. 이 영상이 보이는 것은 **3000 회 동안 우리가 어떻게 정돈되는가** 다.

지형은 rough6 다. 우리 학습 지형 여덟 중 중 여섯이고 omni_gap 과 rails 가 빠진다 (집합 등록이 unseen10 · rough6 둘뿐이다 · sim/eval/terrains.py:TERRAIN_SETS). 그래서 「학습 지형」이라고 뭉뚱그리지 않고 「**학습에 쓴 험지 6 중**」이라고 적는다.

14. 근거 자료

이 절의 sim/, _out/, params/ 등은 저장소 또는 로컬 원자료 경로다. 해당 원자료 전체는 이 제출 파일에 포함되지 않는다. 공개 링크와 재현에 필요한 내부 경로를 구분해 읽어야 한다.

무엇	어디
스weep 원자료 1,182 줄	sim/eval/results/20260928-v2-sweep/sweep_long.csv
난이도 x 속도 요약	같은 폴더 sweep_matrix.md
지형 x 난이도 (속도별)	같은 폴더 sweep_terrain_v*.md

무엇	어디
빠진 칸	같은 폴더 MISSING.md · 0 건
갤러리 v2	/gallery/ 에서 v2 를 고른다
계보 실측	inbox/jay/20260928-v2-mvp/LINEAGE-FACTS.md
멈출 기준 정본	inbox/jay/20260928-v2-mvp/STOPPING-RULE.md · 규칙과 수는 8 절에 본문으로 다 적었다
평가 규격	평가 프로토콜 정본
계보 전체	우리는 왜 여덟 번 방향을 틀었나
지형 카탈로그 스틸 16 장	sim/eval/results/20260928-terrain-shots/stills/ · 퍼기 전후 값은 같은 폴더 index.json
학습 진행 41.3 초 (대군 · 칸당 20 · 배속 곡선)	sim/eval/results/20260929-train-army-ramp/train-army.mp4 · 웹 사본 docs/assets/video/v2/train-army.mp4 · 생성 _out/loop/train_army_ramp.sh · 연결 _out/loop/train_army_ramp_join.py
학습 지형 비교 컷들 (6-6)	sim/eval/results/20260929-traingap/{forward_gap,omni_gap}/flat_army_A_9_topdown.mp4 · 웹 사본 docs/assets/video/v2/train-gap-{forward,omni}.mp4
그 컷을 찍은 것	_out/loop/shoot_traingap.sh · 지형 등록은 sim/eval/terrains.py 의 train_fwdgap · train_omnigap
평지 HUD 를 얻은 것	_out/loop/axis2_hud.py · 원본은 sim/eval/results/20260929-axis2-fall/ 와 20260928-turnfall-clips/ 의 시계열 parquet · 결과는 20260929-axis2-hud/
실패 판을 찾아 찍은 것	_out/loop/shoot_failcuts.sh · 결과는 sim/eval/results/20260929-failcuts/ · 채택한 시드는 그 폴더의 chosen.txt
자리 가설을 기각한 진단	_out/loop/probe_ss_origin.sh · 결과는 sim/eval/results/20260929-ss-origin/
판마다의 전진 거리 (100 판)	sim/eval/results/20260928-devicecheck/v2g2-iter3000-unseen10-d0.5-v1.0-cuda0/generalization_raw.csv
확장 축 2 (v2g2)	sim/eval/results/20260928-v2g2-axis2-ext/v2g2-feetair01-iter3000/probe_manifest.json
목적성 세 화면의 컷 아홉 편	갤러리에서 가져왔다 · NVIDIA 는 gallery/v1/clips/<지형>-<속도>-baseline.mp4 · v1 은 같은 폴더의 -baseline 없는 것 · v2 는 gallery/v2/clips/ · 체크포인트 sha 는 gallery/versions.json 이 적어 둔 것과 대조했다 · 가져온 도구는 _out/loop 밖 임시 스크립트가 아니라 이 표에 경로로 남긴다
지형 열여섯 표의 숫자	gallery/v2/manifest.json 의 evaluations 144 칸 (16 지형 x 3 속도 x 3 판) · 손으로 안 적고 읽어서 생성했다
확장 축 컷 열여덟 편을 찍은 것	_out/loop/shoot_axis2_ext.sh · 판 셋 x 시나리오 여섯 · env 64 · 시드 42 · 영상 env 51 · 원본은 sim/eval/results/20260929-axis2-ext-clips/
그 컷에 HUD 를 얻은 것	_out/loop/axis2_ext_hud.py · axis2_hud.py 의 build_trace 를 그대로 가져다 쓴다 · 결과는 20260929-axis2-ext-hud/ · 울타리마다 속도 축을 같게 맞춘다
NVIDIA · foothold-v1 의 회전 두 칸 (새로 잼)	sim/eval/results/20260929-axis2-ext-clips/{nvidia,v1}/{turn_rest,turn_rev}/per_env.json

FOOTHOLD

원보고서와 근거 자료: FOOTHOLD v2 종합보고서

이 보고서는 전체 프로젝트 중 시뮬레이션 보행 정책의 학습과 평가를 중심으로 MVP 결과를 제시한다. 결과 기준일: 2026.09.29.